

EOS Composition Partial: Implementation Progress

Working record of what is actually implemented

2026-05-17

This document starts as a copy of `notes/eos_composition_partials_implementation_map.md` and should be updated as the implementation changes. The original map and the longer plan remain the starting-point design documents:

```
notes/eos_composition_partials_plan.md
notes/eos_composition_partials_implementation_map.md
notes/eos_composition_partials_implementation_map.pdf
```

The Skye paper source has also been downloaded and unpacked locally for equation checks:

```
notes/skye_arxiv_2104.00691.tar.gz
notes/skye_arxiv_2104.00691/
```

This progress document records what actually lands, what is still open, and how far each implementation piece has moved.

Implementation Progress

`\begin{decisionbox}` **Current checkpoint.**

Dev implementation is in progress on branch `EbF/implicit_diffusion`. The new path is behind dev controls. The source compiled through `./install` before the latest row-scoped EOS-wrapper and Brunt path-average edits; those latest edits have had static checks only. A targeted EOS composition-partial validation plotter now lives in `eos/plotter/composition_partials`. It produces a constrained composition finite-difference CSV, a raw-partial contour CSV, and PDF/PNG contour plots from the default EOS controls. The validation pass fixed an actual HELM μ derivative gap and then exposed a table-EOS gauge issue: the OPAL/SCVH and FreeEOS native X, Z derivatives now map through the same raw XH and residual $Z = 1 - XH - XHe$ coordinates used by `chem_lib:basic_composition_info`. This preserves constrained sink-projected derivatives and the Brunt contraction while making the raw `lnE` and `lnP` columns match the composition coordinates used by the star equations. The production cell path now uses a row-scoped cheaper wrapper that asks the EOS only for `lnPgas` and `lnE` composition rows. The implicit Brunt path defaults back to one face EOS partial call per face; the two-point Gauss composition-path average is available behind the dev `implicit_diffusion_use_brunt_gauss_path` star_job control and is off by default. The Brunt wrapper only requests `chiT, chiRho` composition rows when the optional `d_sigma/dX` Jacobian block is enabled. The cell-centered star EOS composition-partial call now reuses the composition moments already computed by `micro:do_eos_for_cell`. The new `eosDT_get_dxa_rows_with_moments` wrapper builds the raw moment derivative columns once and passes them through `eosDT`, so Skye, HELM, ideal, and the Skye/PC blend-alpha chain rules do not each repeat the same `basic_composition_info` scan on the hot production path. Skye no longer allocates the active-number-fraction Jacobian matrix on each composition-partial EOS call; it computes the one active-number-fraction partial column needed by the existing isotope loop. The isotope residual also now keeps the expensive `d_sigma/dX` cross-species block fully behind `implicit_diffusion_include_dsig_dxa`; when that control is false, the residual no longer loops over all species pairs to add zeros. The `sig_implicit_ad` structure partials are written directly from the AD slot array rather than through the generic full-nvar residual unpacker. The nonlinear structure-Jacobian part of `sig_implicit_ad` is now also behind `implicit_diffusion_include_dsig_structure`, default false. With the nonlinear coefficient controls false, the solver-iteration path still refreshes scalar `sig(:)` values from the current implicit coefficient, but skips the extra coefficient-derivative Jacobian blocks. The FreeEOS/OPAL raw-gauge fix remains in the table wrapper. The local one-last-digit `atm-check_tau` golden-output difference has been updated in `atm/test/test_output`. No MESA model run or EOS table regeneration has been done. When a nonlinear implicit-Dmix Jacobian block is enabled, the solver refreshes the implicit MLT/TDC coefficient while preserving `Dmix_explicit` from the last full mixing-info pass, and then rebuilds `sig(:)` from that consistent split. `\end{decisionbox}`

Active Remaining Work Checklist

Current allowed verification.

Current repo instructions reserve MESA compile and run steps for the user unless explicit permission is given. The current cleanup therefore uses source review, whitespace checks, line scans, and call-chain scans only.

Item	Status	Notes
Compile integration	<code>./install</code> successful	<code>./clean</code> completed earlier. After updating the local <code>atm/test/test_output</code> one-last-digit <code>tau</code> golden value, <code>./install</code> completes successfully through the library checks.
Skye/eosDT compile cleanup	Previous compile clean; latest edits static-only	Fixed the Skye <code>%val</code> function-result selector issue. EOS compiled on the later install passes before the latest Skye Coulomb speed refactors. The latest speed edits have only had static checks because MESA compile/model runs are user-owned.
Skye derivative validation	First plotter pass complete	<code>eos/plotter/composition_partials</code> compares full <code>d_dxa</code> rows with constrained Ridders finite differences. Skye-dominated sweeps are broadly consistent for the rows needed by implicit Brunt; remaining large failures are in the cool HELM/FreeEOS part of the default EOS path, not in the Skye-dominated points. No MESA model/test-suite run is approved in this checkpoint.
Pure PC internal composition partials	Deferred	Not easy; needs PC-local derivatives through MELANGE9 and PC mixing/phase algebra. Blend-alpha derivatives around PC are already wired where available.
Complete Brunt EOS-composition form	Implemented, needs validation	Defaults to one face EOS call for first pressure-composition partials. A two-point Gauss path average of those partials through the composition jump is available as <code>implicit_diffusion_use_brunt_gauss_path</code> = <code>.true.</code> and is off by default. <code>chiX_face</code> is treated as a current-iterate coefficient; second EOS composition derivatives are intentionally out of scope.

Item	Status	Notes
Implicit transport components	Implemented for current policy	<code>Dmix_implicit</code> accepts local <code>mlt_D_ad</code> only in cells whose post-cleanup <code>mixing_type</code> from the last full <code>set_mixing_info</code> pass is <code>convective_mixing</code> or <code>semiconvective_mixing</code> . Cells not promoted to the implicit component keep the normal MESA coefficient in <code>Dmix_explicit</code> . During Newton iterations, <code>set_hydro_vars</code> refreshes the promoted coefficient value after <code>set_mlt_vars</code> while preserving both the full-pass set of implicit zones and <code>Dmix_explicit</code> from the last full mixing-info pass. <code>set_vars_for_solver</code> rebuilds diffusion sigmas every implicit solver iteration; the nonlinear <code>d_sigma</code> controls only decide whether the extra coefficient-derivative Jacobian blocks are filled and used.
Implicit-Dmix residual conservation/sign checks	Static sign check complete	The implemented lower/upper face signs match <code>set_dxdt_mix</code> : <code>dxdt_mix=(fluxp1-flux00)/dm</code> , with <code>flux=-sigma*dX</code> . Runtime validation still needed.
Production cleanup	Started	Normal-path <code>get_convection_sigmas</code> no longer zeros species-sized implicit Jacobian arrays unless <code>implicit_diffusion_flag</code> is true, and <code>set_Dmix_components</code> is only called on the dev implicit path. The dev flag now rejects <code>use_other_brunt</code> , <code>use_other_brunt_smoothing</code> , and <code>use_other_mlt_results</code> , which would otherwise bypass the internal Brunt/MLT derivative path. The implicit face-EOS Brunt path is now skipped when <code>use_Ledoux_criterion</code> is false, and <code>implicit_brunt</code> reuses scratch storage per OpenMP worker instead of allocating per face. Further review still needed.

Area		Status	Current implementation state	Next update trigger
Composition helper and moment derivatives		Started	Added private EOS helper for abar , zbar , z2bar , z53bar , Ye , mass-correction derivatives, and active-set number-fraction derivatives. Skye and ideal now call it; eosDT PC blend partials use the abar , zbar chain rule.	Update when remaining component blend-alpha routines use moment derivatives.
Skye ideal-ion composition partials	Implemented for current path		Added active-set number-fraction chain rule for ideal-ion free energy and ion-offset derivatives, then pack all EOS result rows through the Skye thermodynamic identities. The production row-scoped path now computes only requested rows and reuses active-number-fraction helper results.	Update after focused derivative checks or if additional Skye rows are promoted into the production path.
Skye electron composition partials		Started	Extended HELM electron helper to return the free-energy derivative with respect to Ye ; Skye maps it through dYe/dxa and fills i_eta . The helper now fills the full third-order F_dye object using the explicit F_e=Y_e f(T,Y_e\rho) chain rule; fourth density derivatives of the quintic HELM basis were added so the rho,rho,rho slot is also analytic.	Update when remaining electron scalar derivatives, if any, are audited against finite differences.

Area	Status	Current implementation state	Next update trigger
Skye Coulomb/phase composition partials	Implemented; speed-refactored for hydro rows	Added the Skye-local skye_composition_ad companion type and wired an analytic companion path through nonideal_corrections_dx . The path covers liquid/solid OCP terms, screening terms, liquid and solid mixing corrections, electron exchange-correlation, mixing entropy, Skye's Gamma-limit extrapolation, and the phase softmin. The production hydro-row path now uses basis derivatives in Ye , abar , and active Skye number fractions, reuses the base hard-branch Coulomb AD result for off-transition dYe/dabar , and reuses cached OCP leaf values for batched dYA . Phase-transition cells and phase/latent rows still use the full companion evaluation.	Update after compile-level review and focused derivative checks across liquid, solid, extrapolated, and phase-transition states.
eosDT blend composition partials	Started	Do_Blend still carries selector composition derivative plumbing for diagnostics, but composition rows are currently blended at fixed EOS selector weights. This avoids adding dalpha/dxa*(res_1-res_2) terms from numerical EOS transition functions to the physical composition Jacobian. PC Gamma-limit, OPAL/SCVH high-Z HELM reduction, and the full Skye polygon still have alpha derivative code available, but those selector derivatives are not used in production composition rows while the fixed-weight blend policy is active.	Update if selector composition derivatives are re-enabled or replaced by a potential-level blend.

Area	Status	Current implementation state	Next update trigger
Complete Brunt EOS-composition form	Implemented, needs validation	Added an implicit Brunt path in <code>star/private/implicit_brunt_f0</code>. Under <code>implicit_diffusion_flag</code> and <code>use_Ledoux_criterion</code>, <code>hydro_vars:set_hydro_vars</code> calls this path even during solver iterations, so the Brunt composition term is updated from face EOS composition partials instead of the old two-composition pressure difference. When Ledoux is off, the path keeps <code>gradL_composition_term_ad(:)</code> explicitly zero and skips the expensive face EOS calls. This is the complete EOS-composition form, not a reduced mean-molecular-weight derivative. Added AD storage <code>brunt_B_ad(:)</code> and <code>gradL_composition_term_ad(:)</code>. Added species-indexed <code>d_brunt_B_dxa_m1(:, :)</code> and <code>d_brunt_B_dxa_00(:, :)</code>; these include the direct face pressure-partial contraction, the cell-pressure denominator, face <code>chi_T/chiRho</code> terms, and the direct mass-correction derivative when <code>implicit_diffusion_include_dsig_dxa</code> is enabled. The face EOS wrapper now accepts the already computed face/path composition moments, so it avoids a second moment pass inside <code>eosDT</code>. The shared <code>set_grads</code> smoothing step is now skipped only for the implicit Ledoux path, so implicit Brunt remains unsmoothed while non-Ledoux runs keep the normal Brunt path. The face EOS	Update when face-coefficient <code>brunt_f0</code> changes or if second EOS composition derivatives become an explicit goal.

Area	Status	Current implementation state	Next update trigger
Coupled diffusion residual contribution	Started	Existing isotope residual is still the only species equation. Added the implicit-Dmix Jacobian contribution inside <code>star/private/hydro_chem_eqns.f90</code> : the rate remains <code>dxdt_mix</code> , while the fixed-coefficient composition solve is implicit whenever <code>implicit_diffusion_flag</code> is true. The nonlinear structure partials from <code>sig_implicit_ad(:)</code> are now gated by <code>implicit_diffusion_include_dsig_structure</code> , default false. The high-gain cross-species terms from <code>d_sig_dxa_m1(:, :)</code> and <code>d_sig_dxa_00(:, :)</code> are gated by <code>implicit_diffusion_include_dsig_dxa</code> and <code>use_Ledoux_criterion</code> , also default false while the convective-boundary coupling is validated. The upper face uses <code>shift_p1</code> , following the standard MESA AD stencil convention. Static sign review matches <code>set_dxdt_mix</code> .	Update when runtime validation is added or when microscopic diffusion is moved into this residual.

Area	Status	Current implementation state	Next update trigger
Nuclear network composition partials	Audited for current concern	basic.net does not use the hardwired approx21 implementation. It enters the generic net_derivs:get_derivs path and exports d_dxdt_nuc_dx through the molar-abundance chain rule. The hardwired net_approx21 path is only enabled by net files containing an approx21(...) directive. One separate generic-net caveat remains: screened rates are differentiated with respect to T and rho , but their composition dependence through screening/mean composition is treated as fixed in d_dxdt_nuc_dx . That is a long-standing approximate Newton Jacobian and could produce finite-difference disagreement in screened regimes, but it is not an approx21 issue for basic.net .	Add a targeted local dxdt_nuc(he4) finite-difference diagnostic if the He4 equation remains suspicious after the implicit-Dmix refresh is tested.

Area	Status	Current implementation state	Next update trigger
Dmix_explicit / Dmix_implicit split	Current policy implemented; needs runtime validation	<code>Dmix_implicit(:)</code> is <code>auto_diff_real_star_order1</code>. <code>Dmix_explicit(:)</code> is the semi-implicit coefficient held fixed through Newton iterations. <code>turb_support:do1_mlt_eval</code> returns <code>D_ad</code>, and <code>turb_info:do1_mlt_2</code> stores it in <code>s%mlt_D_ad(k)</code>. <code>implicit_Dmix:set_Dmix_components</code> puts that local <code>mlt_D_ad</code> into <code>Dmix_implicit</code> only in cells whose post-cleanup <code>mixing_type</code> from the last full mixing-info pass is <code>convective_mixing</code> or <code>semiconvective_mixing</code>; <code>Dmix_explicit</code> stores the non-implicit part of the ordinary MESA total on full mixing-info passes. The total is always rebuilt as <code>D_mix = Dmix_implicit%val + Dmix_explicit</code>, preserving the full-pass MESA coefficient while keeping the stored components non-negative. During solver iterations, <code>hydro_vars:set_hydro_vars</code> calls <code>set_Dmix_components(s,.false.)</code> after the current <code>set_mlt_vars</code> call when the full <code>set_mixing_info</code> pass is skipped; this refreshes the promoted coefficient value while leaving the full-pass set of implicit zones and <code>Dmix_explicit</code> fixed from the last full mixing-info pass. <code>solver_support:set_vars_for_solver</code> calls <code>mix_info:get_convection_sigmas</code> every implicit solver iteration so the residual uses the current total coefficient. Structure partials from <code>sig_implicit_ad</code> are	Update when separate controls are added for promoted transport terms, post-processing contributions are tracked separately, or the custom MLT hook becomes composition-AD aware.

Area	Status	Current implementation state	Next update trigger
Internal full- d_dxa EOS path	Implemented for current production/diagnostic split	Added eosDT_get_full_dxa and star-side get_eos_full_dxa for diagnostics/plotter use, plus eosDT_get_dxa_rows, eosDT_get_dxa_rows_with_moments, get_eos_star_dxa, get_eos_brunt_dxa, and get_eos_brunt_dxa_with_moments for production row-scoped requests. micro:do_eos_for_cell now uses the moment-aware cheaper star wrapper when include_eos_composition_partials is true, so the star solve gets only lnPgas and lnE rows and does not recompute the composition moments it already stored in s% X, s% abar, s% zbar, s% ye, and related arrays. The implicit Brunt face wrapper now reuses the face/path moments computed in implicit_brunt. Under that flag s% d_eos_dxa is still allocated with all num_eos_basic_results rows for stable indexing; otherwise it remains the legacy two-row storage. The full/row request is threaded through Get_eosDT_Results into the component EOS interface so expensive Skye companion derivatives, ideal full-row packing, eosDT blend-alpha composition terms, and composition-moment helper calls are paid for only on requested rows. Timing counters now split dxa cost into moment, blend, check, component, table, and Skye ideal/Coulomb/pack work.	Update when the HELM/FreeEOS derived-row audit, row-sized internal scratch cleanup, or pure-PC path is addressed.
		Get_eosDT_Results	

Area	Status	Current implementation state	Next update trigger
EOS composition-partial controls	Started	Added dev <code>include_eos_composition_controls</code> flag. Flag off means current EOS wrapper behavior, not suppressing existing partials. <code>implicit_diffusion_flag</code> no longer forces this flag by itself; the hydro EOS partials are forced on only when <code>implicit_diffusion_include_dsig_dxa</code> is also true, because that optional Jacobian block is the part that consumes EOS composition rows. The finite-difference <code>fix_d_eos_dxa_partials</code> fallback is skipped on the dev full-partial path. The validation plotter deliberately uses a default EOS handle, so the first pass tests the default eosDT selection and records the EOS fractions instead of tuning a special EOS-control setup.	Update when EOS call <code>update_partials</code> using component-level analytic partials or when component-forced validation is added.
Verification	Prior compile/plotter pass complete; latest speedups static-only	<code>./install</code> completed successfully after the HELM mu derivative fix, before the latest Skye Coulomb speed refactors. The EOS plotter writes the 1D finite-difference CSV, a 2D contour CSV, 14 contour pages as both PDF and PNG, and a combined report at <code>figures/all_composition_partial_contours.pdf</code> . The latest timing-driven Skye speedups have only had static checks here; user timing runs show the expected EOS-speed reduction. Brunt/Dmix runtime validation is still pending.	Update after the next approved compile/install, Brunt/diffusion numeric validation, or HELM/FreeEOS derived-row audit.

The implicit diffusion split is

$$D_k = \text{val}(D_{k,\text{ad}}^{\text{imp}}) + D_k^{\text{expl}}.$$

In code, `D_mix(:)` is the ordinary real MESA total coefficient, while `Dmix_implicit(:)` carries the AD derivatives for the promoted coefficient and `Dmix_explicit(:)` is the semi-implicit real coefficient held fixed during Newton iterations.

The AD coefficient comes from the current MLT/TDC solve:

$$D_{k,\text{ad}}^{\text{mlt}} \leftarrow \text{turb_support:do1_mlt_eval}.$$

`turb_info:do1\_mlt\_2` stores this as `s% mlt_D_ad(k)`. The promoted component is

$$D_{k,\text{ad}}^{\text{imp}} = \begin{cases} D_{k,\text{ad}}^{\text{mlt}}, & \text{mixing_type_full}(k) \in \{\text{convective_mixing}, \text{semiconvective_mixing}\}, \\ 0, & \text{otherwise.} \end{cases}$$

Here `mixing_type_full` means the post-cleanup `mixing_type` left by the last full `mix_info:set_mixing_info` pass. The coefficient value and derivatives come from the current MLT/TDC `mlt_D_ad`, but the allowed implicit zones are held to the last full-pass post-cleanup convective/semiconvective set. This means a zone that becomes formally convective in the current MLT/TDC solve during a Newton iteration does not enter `Dmix_implicit` until the next full `set_mixing_info` pass. That is deliberate: changing which zones are mixed also requires the ordinary MESA cleanup, pruning, boundary, overshoot, minimum mixing, rotation, and user-zeroing edits. The solver-iteration path only updates coefficients within the set of implicit zones from the last full pass.

`implicit_Dmix:set_Dmix_components` has two explicit call modes:

- `set_Dmix_components(s,.true.)` is used at the end of a full `mix_info:set_mixing_info` pass. It refreshes both `Dmix_implicit` and `Dmix_explicit`, then rebuilds `D_mix`. The split preserves the ordinary MESA total coefficient from that full pass. For promoted local MLT/TDC cells, `Dmix_explicit` stores the non-implicit part of the total. If later full-pass processing made the MESA total smaller than the local MLT/TDC value, the promoted AD coefficient is scaled down to the total so the two stored components remain non-negative. For cells not promoted to `Dmix_implicit`, the current ordinary MESA `D_mix` is stored in `Dmix_explicit`.
- `set_Dmix_components(s,.false.)` is used inside solver iterations after `set_mlt_vars`, because the full `set_mixing_info` pass is skipped there. It refreshes only the value and derivatives of `Dmix_implicit` inside the stored full-pass set of implicit zones; `Dmix_explicit` remains the value from the last full mixing-info pass. The routine then rebuilds total `D_mix`.

`mix_info:get_convection_sigmas` then rebuilds

$$\sigma_k = \frac{\text{mix_factor } D_k (4\pi r_k^2 \rho_{\text{face},k})^2}{\Delta m_{\text{avg},k}},$$

so the residual value uses total `D_mix`. When `implicit_diffusion_include_dsig_structure` is true, the same routine also builds the implicit AD version

$$\sigma_{k,\text{ad}} = \frac{\text{mix_factor } D_{k,\text{ad}}^{\text{imp}} (4\pi r_{k,\text{ad}}^2 \rho_{\text{face},k,\text{ad}})^2}{\Delta m_{\text{avg},k}},$$

while keeping `sig_implicit_ad(k)%val = sig(k)` so the residual value still uses the full explicit-plus-implicit coefficient. If `implicit_diffusion_include_dsig_structure` is false, these structure derivatives are not filled or consumed. The Ledoux `implicit_diffusion_include_dsig_dxa` block separately controls the cross-species coefficient derivatives.

Current call order:

- Outside Newton iterations, `hydro_vars:set_hydro_vars` calls `set_mlt_vars`, then the full `mix_info:set_mixing_info` pass. At the end of that full pass, `implicit_Dmix:set_Dmix_components(s,.true.)` builds `Dmix_implicit`, `Dmix_explicit`, and the total `D_mix`.
- Before the abundance solve, `mix_info:get_convection_sigmas` builds scalar `sig(:)` from total `D_mix`.
- Inside Newton iterations, `solver_support:set_vars_for_solver` calls `hydro_vars:set_hydro_vars` with the full mixing-info pass skipped. For implicit diffusion plus Ledoux, `do_implicit_brunt_B` is still called so current MLT/TDC sees the current Brunt composition term. After `set_mlt_vars`, `set_Dmix_components(s,.false.)` refreshes only `Dmix_implicit` and leaves `Dmix_explicit` fixed from the last full pass.
- `solver_support:set_vars_for_solver` then calls `mix_info:get_convection_sigmas`, so `sig(:)` uses the current total coefficient while its optional derivative blocks come only from `Dmix_implicit`.

Full-pass mixing cleanup caveat:

- The full `mix_info:set_mixing_info` pass first copies current MLT/TDC results into `mixing_type`, `D_mix`, `cdc`, and `conv_vel`.
- It then applies the ordinary MESA cleanup/edit sequence, including `remove_tiny_mixing`, `remove_mixing_singletons`, gap closing, embedded semiconvection cleanup, predictive mixing, overshoot, minimum mixing, rotation, and user zeroing regions.
- `locate_convection_boundaries` can also prune a small non-burning convective region by setting `D_mix = 0`, `cdc = 0`, `conv_vel = 0`, and `mixing_type = no_mixing`.
- Only after these full-pass edits does `set_Dmix_components(s,.true.)` split the post-cleanup MESA total into `Dmix_implicit` and `Dmix_explicit`. Thus a full-pass cleanup decision does affect the stored implicit component.
- During Newton iterations the full `set_mixing_info` cleanup sequence is skipped. The solver refresh recomputes MLT/TDC and then calls `set_Dmix_components(s,.false.)`; after the 2026-05-17 correction, this refresh uses the last full-pass post-cleanup `mixing_type` to choose which zones may use `Dmix_implicit`, while taking the promoted coefficient value from current `mlt_D_ad`. This choice affects the residual during Newton through the rebuilt `D_mix` and `sig(:)`; it is not just a final post-step cleanup.

Raw-current-MLT zone-selection failure mode now blocked:

1. A full `set_mixing_info` pass finds a one-zone or very thin convective region. The patchy-convection cleanup removes it, leaving post-cleanup `D_mix(k)=0` and `mixing_type(k)=no_mixing`. The full split therefore stores `Dmix_implicit(k)=0` and keeps the total zero.
2. In a later Newton iteration of the same step, the current Brunt/MLT result at that zone flips back to `mlt_mixing_type(k)=convective_mixing`. Because the full cleanup sequence is skipped, `set_Dmix_components(s,.false.)` can put the current `mlt_D_ad(k)` back into `Dmix_implicit(k)`.
3. The rebuilt `D_mix(k)=Dmix_implicit(k)%val+Dmix_explicit(k)` can then turn mixing back on during the residual evaluation, even though the last full mixing-info pass had deliberately pruned that patch.

The 2026-05-17 correction closes this particular turn-on path by using the post-cleanup full-pass `mixing_type` to choose implicit zones during solver iterations. The tradeoff is the opposite failure mode: a zone that was minimum, overshoot, rotation, or no mixing in the last full pass cannot newly join the implicit component inside the same Newton solve. It can join on the next full `set_mixing_info` pass, which normally occurs at the start of a new step attempt or retry. This is a conservative outer-loop treatment of a discontinuous zone selection, not a fully implicit treatment of zone-selection changes. If runtime validation still shows boundary instability, the next more restrictive option is to store a full-pass local maximum for `Dmix_implicit` and cap the solver-iteration refresh. That needs a separate design decision because it may suppress legitimate Newton updates when a convective boundary is physically moving.

Could solver iterations rerun the start-of-step mixing checks? In principle yes, but the full `set_mixing_info` pass is not just a local cleanup of current MLT/TDC. It also rebuilds convective and mixing boundaries, may apply predictive mixing, overshoot, minimum mixing, rotation, zeroing regions, and user hooks, and then refreshes the explicit/implicit component split. Calling that full pass inside every Newton iteration would remove the one-step lag in the set of zones allowed into `Dmix_implicit`, but it would also let discontinuous start/stop decisions change during the Newton solve without a Jacobian for those decisions. A safer future design would be a restricted solver-iteration check pass that mirrors only the local no-mixing/minimum/zero edits needed to keep `Dmix_implicit` consistent, while leaving nonlocal boundary construction and post-processing for the full `set_mixing_info` pass.

2026-05-17 correction to the design direction: using the last full-pass `mixing_type` as the solver-iteration gate is too conservative because it can lag both ways. If a full pass pruned a zone to `no_mixing`, current MLT/TDC cannot turn the implicit coefficient on during Newton; if a full pass allowed implicit convection, current controls or state changes may not turn it off until the next full pass. MESA also uses `no_mixing` for two different things: “MLT currently found no mixing” and “a control or cleanup forced zero mixing.” The cleaner policy is therefore to make the solver-iteration implicit coefficient follow current `mlt_mixing_type` and current `mlt_D_ad` directly, while using a separate, explicit predicate only for true forced exclusions. Start-of-step cleanup such as tiny/singleton/pruned convective regions should not by itself prevent `Dmix_implicit` from becoming nonzero later in the same Newton solve.

The high-temperature core-collapse test on 2026-05-17 shows the same pattern: large isotope corrections and retries occur on five-zone `mix_type` stencils such as 17731, 37733, 73311, and 71177, with the largest corrections often in Si/S/O/C near the transition between convective, semiconvective, and minimum-mixing cells. This does not by itself prove the implicit-`Dmix` gate is the cause, but it is the expected failure signature if current MLT/TDC wants a different implicit coefficient than the one implied by the last full `set_mixing_info` result. The next diagnostic should print, for the failing zones and their neighbors, `mixing_type`, `mlt_mixing_type`, `D_mix`, `Dmix_implicit%val`, `Dmix_explicit`, `mlt_D`, `mlt_D_ad%val`, and `sig` during each solver iteration.

2026-05-17 design audit:

- The old optional flag name did not say whether the call was a full mixing-info refresh or a solver-iteration refresh. The code now uses `update_explicit_Dmix`: true means refresh both `Dmix_explicit` and `Dmix_implicit`; false means refresh only `Dmix_implicit`.
- It would be incorrect to zero `Dmix_explicit` during a solver iteration. The solver-iteration path does not do that. During a full mixing-info pass, `Dmix_explicit` stores the part of the ordinary MESA total not represented by `Dmix_implicit`.
- The routine name now reflects that it manages the split and rebuilds total `D_mix`; it is `set_Dmix_components`, not an implicit-only updater.
- `D_mix(:)` remains the ordinary real MESA coefficient used by existing mixing and sigma code. The Newton derivatives of the promoted component are carried separately through `Dmix_implicit(:)`, `sig_implicit_ad(:)`, and, when enabled, `d_sig_dxa_m1/00(:, :)`.

The isotope equation is still the existing one,

$$\left(\frac{dX_{j,k}}{dt} \right)_{\text{mix}} = \frac{\sigma_k(X_{j,k-1} - X_{j,k}) - \sigma_{k+1}(X_{j,k} - X_{j,k+1})}{\Delta m_k},$$

and the residual/Jacobian use the current-iteration scalar `sig(:)`. The optional nonlinear coefficient blocks add the structure and cross-species derivatives of that coefficient.

The optional composition dependence of the implicit diffusion coefficient enters through

$$\frac{\partial R_{j,k}}{\partial X_{\ell,k}} \supset \frac{1}{\Delta m_k S_{j,k}} \left[\frac{\partial \sigma_k}{\partial X_{\ell,k}} (X_{j,k-1} - X_{j,k}) - \frac{\partial \sigma_{k+1}}{\partial X_{\ell,k}} (X_{j,k} - X_{j,k+1}) \right],$$

where $S_{j,k}$ is the chemistry equation scale. This term is powerful near convective boundaries because changing a trace isotope can move the Ledoux/MLT stability coefficient and therefore the He4 diffusion flux. The observed main-sequence failure pattern

$$\max |R| = \text{equ_he4}, \quad \max |\delta X| = \text{o16}$$

is consistent with this cross-species `d_sigma/dX` coupling: the Newton matrix can try to change O16 in the neighboring cell to reduce the He4 flux residual. For this reason `implicit_diffusion_include_dsig_dxa` defaults false. With the default setting the implicit diffusion path keeps the usual same-species implicit composition solve and refreshes only the scalar coefficient values inside Newton iterations.

The main-sequence retry tests showed that turning off this cross-species block does not remove the `equ_he4` failure. That points back upstream to the implicit Brunt value used by MLT/TDC:

$$B_k^{\text{imp}} = \frac{\Delta \ln P_X}{\Delta \ln P_{\chi_T}}.$$

For table EOS components such as FreeEOS and OPAL/SCVH, the numerator was previously a one-point face linearization of a native table surface,

$$\Delta \ln P_X = (q_X \Delta X_H + q_Z \Delta Z)_{\text{face}}, \quad q = \ln P_{\text{gas}}.$$

This is analytic in the table-interpolation sense, but it is only a midpoint directional derivative through the composition jump. If q_X or q_Z is noisy near a table/blend transition, implicit diffusion amplifies that noise through the stability condition even when `d_sigma/dX` is disabled.

An optional follow-up can replace the one-point face contraction by a short path average,

$$\Delta \ln P_X = \int_0^1 [q_X(X(\lambda), Z(\lambda)) \Delta X_H + q_Z(X(\lambda), Z(\lambda)) \Delta Z] d\lambda,$$

evaluated by two-point Gauss quadrature. This keeps analytic table derivatives but costs two extra row-scoped face EOS calls, so `implicit_diffusion_use_brunt_gauss_path` defaults false. A diagnostic/secant mode remains a possible debugging tool,

$$\Delta \ln P_X = \ln P_{\text{gas}}(X_k, Z_k) - \ln P_{\text{gas}}(X_{k-1}, Z_{k-1}),$$

at fixed face T, ρ . That would not be a partial-derivative formulation, but it could still isolate whether a table derivative is the unstable ingredient.

For FreeEOS/OPAL/SCVH specifically, the optional path-average stays in the native table coordinates $X = X_{\text{H}}$, $Z = 1 - X_{\text{H}} - X_{\text{He}}$, and $Y = 1 - X - Z$. A path-averaged Brunt numerator would use

$$X(\lambda) = X_0 + \lambda \Delta X, \quad Z(\lambda) = Z_0 + \lambda \Delta Z, \quad Y(\lambda) = 1 - X(\lambda) - Z(\lambda),$$

so the composition constraint is preserved at every quadrature point. The stored raw isotope columns do not need to sum to zero or one. They are a gauge for the constrained derivative. With the current table mapping,

$$q_{\text{H}} = q_X - q_Z, \quad q_{\text{He}} = -q_Z, \quad q_{\text{metal}} = 0,$$

and any constrained composition difference gives

$$\sum_j q_j \Delta X_j = q_X \Delta X_{\text{H}} + q_Z \Delta Z, \quad \sum_j \Delta X_j = 0.$$

Thus `implicit_brunt` does not need to choose a sink species for the face contraction; the sink only appears in finite-difference diagnostics or other places that explicitly convert raw columns into $n - 1$ independent variables.

Potential-Level EOS Blend Design

Longer-term FreeEOS/Skye consistency option.

If the FreeEOS side can expose a thermodynamically consistent Helmholtz free energy, then the cleanest EOS transition would be to blend the thermodynamic potential and derive the EOS result rows from that single blended potential. This is not the current eosDT implementation. The current blend layer in `eos/private/eosdt_support.f90:Do_Blend` blends already-packed EOS result rows and their row derivatives. Skye is already internally potential-based through `eos/private/skye_thermodynamics.f90:thermodynamics_from_free_energy`, but the FreeEOS/OPAL/SCVH table path currently reaches eosDT as result rows and table-coordinate derivatives through `eos/private/eosdt_eval.f90:get1_for_eosdt`.

For a specific Helmholtz free energy per gram $f(\rho, T, \mathbf{X})$, the thermodynamic rows would be derived from

$$P = \rho^2 \left. \frac{\partial f}{\partial \rho} \right|_{T, \mathbf{X}}, \quad s = - \left. \frac{\partial f}{\partial T} \right|_{\rho, \mathbf{X}}, \quad e = f + Ts.$$

A potential-level blend would define

$$f_{\text{blend}} = \alpha(\rho, T) f_{\text{Skye}} + [1 - \alpha(\rho, T)] f_{\text{FreeEOS}},$$

where the composition derivative of the numerical selector remains fixed:

$$\left. \frac{\partial \alpha}{\partial X_j} \right|_{\rho, T} = 0.$$

Then

$$\left. \frac{\partial f_{\text{blend}}}{\partial X_j} \right|_{\rho, T} = \alpha \left. \frac{\partial f_{\text{Skye}}}{\partial X_j} \right|_{\rho, T} + (1 - \alpha) \left. \frac{\partial f_{\text{FreeEOS}}}{\partial X_j} \right|_{\rho, T}.$$

All requested rows, including $\ln P_{\text{gas}}$, $\ln E$, χ_T , χ_ρ , Γ_1 , and ∇_{ad} , would then be packed by the same thermodynamic identities from f_{blend} . This is stronger than blending the already-derived rows,

$$q_{\text{blend}} = \alpha q_{\text{Skye}} + (1 - \alpha) q_{\text{FreeEOS}},$$

because the pressure, energy, entropy, and heat-capacity rows would come from one differentiable potential instead of separate blended output surfaces.

The practical requirements are substantial:

1. FreeEOS would need to return f_{FreeEOS} in the same units and zero point convention as Skye, or the transition would carry an arbitrary offset into entropy and energy.
2. The FreeEOS side would need enough ρ, T derivatives of f to pack the EOS rows already used by star and the validation plotter. Skye carries these derivatives through the AD object used by `thermodynamics_from_free_energy`.

3. Composition partials would need to be in the same constrained composition basis before the potentials are blended. A row-constant gauge projection,

$$D_j q \leftarrow D_j q - \frac{\sum_i X_i D_i q}{\sum_i X_i},$$

remains useful for comparing or validating component partials, but a true potential blend should apply the common basis at the f_{X_j} level.

4. The blend selector can keep its ρ, T derivatives, because those are part of the numerical transition in state space. Its composition derivatives should remain fixed or absent unless the selector itself is promoted to physical EOS content.

This would be a separate design from the current row-level cleanup. The near-term FreeEOS/Skye work should first diagnose whether the current component $\ln E$ and $\ln P_{\text{gas}}$ composition partials disagree sharply at the transition. If they do, the next incremental fix is a shared constrained-basis projection before row-level blending. A potential-level blend is the cleaner thermodynamic endpoint, but it should only be attempted after confirming that FreeEOS can expose the required free-energy object and derivatives through the MESA eosDT interface.

Current Performance Notes

New Implementation Detail

The cell-centered `include_eos_composition_partials` path does not intentionally make an extra EOS call per cell. `micro:do_eos_for_cell` switches the normal `get_eos` call to `get_eos_full_dxa`, and `hydro_eqns` skips the old `fix_d_eos_dxa_partials` finite-difference path when analytic composition partials are enabled. The current cost comes from what is requested inside that single EOS call: the validation implementation asks every component for `d_dxa(1:num_eos_basic_results,1:species)`, even though the normal star solve only needs the legacy rows

$$\ln E, \quad \ln P_{\text{gas}},$$

and implicit Brunt usually needs only the face pressure-composition row

$$\ln P_{\text{gas}}$$

unless the optional `d_sigma/dX` block is enabled.

The production direction is now row-scoped EOS composition partials, not more cell EOS calls. The new internal API requests the needed row list:

$$\{\ln E, \ln P_{\text{gas}}\} \quad \text{for cell-centered star partials,}$$

and

$$\{\ln P_{\text{gas}}\} \quad \text{or} \quad \{\ln P_{\text{gas}}, \chi_T, \chi_\rho\} \quad \text{for implicit Brunt,}$$

with a separate full-row diagnostic mode for the plotter and solver-partial tests. This keeps the old “one EOS evaluation gives the cell thermodynamics” model, but avoids computing unrelated composition rows.

The implemented wrappers are:

- `eosDT_get_dxa_rows`, which passes `dxa_rows(:)` through `Get_eosDT_Results`;
- `get_eos_star_dxa`, which requests only `i_lnPgas` and `i_lnE`; and
- `get_eos_brunt_dxa` and `get_eos_brunt_dxa_with_moments`, which request `i_lnPgas` only, or add `i_chiT` and `i_chiRho` when `implicit_diffusion_include_dsig_dxa` is true.

The older full wrapper, `get_eos_full_dxa`, remains for diagnostics and the plotter. The eosDT blend layer also honors `dxa_rows(:)`, so blend-alpha composition terms are only formed for requested rows.

`implicit_diffusion_flag` no longer forces the hydro cell-centered `include_eos_composition_partials` path by itself. It is forced on only when `implicit_diffusion_include_dsig_dxa` is true, because that optional cross-species `d_sigma/dX` block is the implicit-diffusion path that requires EOS composition rows in the hydro Jacobian.

The current expensive pieces are:

1. cell-centered EOS calls through `micro:do_eos_for_cell` when `include_eos_composition_partials` is true, and
2. face-centered EOS calls through `implicit_brunt:get_implicit_brunt_B` when implicit diffusion also needs the Ledoux composition term.

For non-Ledoux runs there is no Brunt composition contribution to promote into MLT/TDC, so `hydro_vars:set_hydro_vars` now avoids the face EOS path unless both flags are true:

$$\text{implicit_diffusion_flag} \wedge \text{use_Ledoux_criterion}.$$

When `Ledoux` is false, `set_implicit_gradL_composition_term_ad` still runs and sets

$$\nabla_{L,\text{comp},k}^{\text{ad}} = 0,$$

so the MLT/TDC AD path cannot reuse an old composition term.

The implicit diffusion fixed-coefficient solve does not need the nonlinear sigma Jacobian storage. `mix_info:get_convection_sigmas` now clears and updates `sig_implicit_ad(:)` only when `implicit_diffusion_include_dsig_structure` is true, and clears and updates `d_sig_dxa_m1/00(:, :)` only when `implicit_diffusion_include_dsig_dxa` is true and `Ledoux` is active. This keeps the default implicit diffusion path from paying the `species*nz` cross-species storage cost when that Jacobian block is disabled. The isotope equation assembly in `hydro_chem_eqns` uses the same gates, so it also avoids copying `sig_implicit_ad` objects inside the species loop when the structure sigma Jacobian is disabled.

For `Ledoux` runs, `implicit_brunt` still intentionally asks the EOS for face composition partials. The local cleanup is that `xa_face`, `xa_path`, `chiX_face`, and `d_eos_dxa` are allocated once per OpenMP worker and reused across the worker's face loop. The default Brunt numerator uses one row-scoped face-value EOS call,

$$N_{\text{EOS}}^{\text{face}} \simeq N_{\text{face}},$$

per implicit Brunt refresh. If `implicit_diffusion_use_brunt_gauss_path = .true.`, the Brunt numerator adds two row-scoped path-sample calls that request only `i_lnPgas`, giving $N_{\text{EOS}}^{\text{face}} \simeq 3N_{\text{face}}$. That mode is for diagnostics or difficult table-EOS transitions, not the default performance path.

One allocation cleanup is in `eosDT_eval:combine_for_eosdt`: the explicit heap `allocate/deallocate` for component `d_dxa` blend scratch arrays has been removed. The scratch arrays are still full `nv` by `species` work arrays because the component EOS interface still has a full-row `d_dxa` argument; a true row-sized blend scratch would require a larger interface change. That larger interface change is deferred: internally, a row-scoped call such as

$$\{\ln P_{\text{gas}}, \ln E\}$$

could carry only an $n_{\text{rows}} \times n_{\text{species}}$ derivative block through the component and blend stack, then expand or scatter into the legacy `num_eos_basic_results` indexing only at the public API boundary. This would reduce clearing and memory traffic, but it touches many component interfaces in `eos/private/eosdt_eval.f90`, so it should be done only after the physics of the composition partials is settled. Another is in `skye:get`: the active-number-fraction derivative used by the Coulomb composition loop is now computed one isotope column at a time, avoiding allocation of a `relevant_species` by `species` matrix on each Skye composition-partial call. The ideal-ion composition path in `skye_ideal` also computes a single active-number-fraction derivative column at a time and caches the $\log(n_j/n_{\{Q,j\}})$ AD term for reuse in the species loop, avoiding the full active-fraction Jacobian allocation and repeated logarithms. The Skye Coulomb companion path also skips phase and latent-heat companion derivatives unless the requested EOS rows include `i_phase`, `i_latent_ddlnT`, or `i_latent_ddlnRho`. The production star wrapper normally requests only `i_lnPgas` and `i_lnE`, so this avoids carrying the soft phase selector and latent derivatives through every isotope when those rows are not used. The remaining large Skye cost was the per-isotope Coulomb companion call. This has been refactored into a compact basis:

$$\frac{\partial F_{\text{coul}}}{\partial X_j} = F_{\text{coul}, Y_e} \frac{\partial Y_e}{\partial X_j} + F_{\text{coul}, \bar{A}} \frac{\partial \bar{A}}{\partial X_j} + \sum_{i \in A_{\text{Skye}}} F_{\text{coul}, y_i} \frac{\partial y_i}{\partial X_j}.$$

Here A_{Skye} is the active Skye Coulomb species set. The full companion algebra is still used once for Y_e and once for $\bar{A}$, but the active-number-fraction part is now evaluated in one batched `dYA` call. Each network isotope column is then assembled by linear combination. This changes the Coulomb companion cost from roughly `species` full companion calls per cell to two full companion calls plus one batched active-fraction pass. For large networks with many trace isotopes below `mass_fraction_limit_for_Skye`, this should be the first genuinely large speedup in the EOS partial path. The assembly in `skye:get` skips exact zero coefficients in this linear combination. That is algebraically identical to the equation above, but avoids IEEE 0*NaN contamination from an unused basis derivative. Non-finite requested Skye composition rows still return `ierr` before the rows are blended into `eosDT` or stored by `star`. The active-number-fraction basis calls now use a batched Skye Coulomb path: when the perturbation is only dy_i , `skye_coulomb` evaluates each OCP leaf fit once and reuses those values for all active number-fraction columns. The liquid and solid mixing corrections are still differentiated analytically, and the Y_e and $\bar{A}$ basis calls still use the full companion path because their perturbations enter r_s , Γ_e , and the unit conversion. This preserves the same chain rule while avoiding repeated OCP sums for each active y_i column.

There is no hydro-solve mass-fraction cutoff that zeroes returned EOS composition-partial columns. `mass_fraction_limit_for_Skye` only defines the active species included in Skye's Coulomb mixture; raw `d_dxa(:, j)` columns are still returned for every network isotope through $\bar{A}$, Y_e , ion offsets, and table-EOS coordinates. The default is `mass_fraction_limit_for_Skye = 1d-4` in `eos/defaults/eos.defaults`.

Star timing now also records the composition-partial EOS subpath when `s% doing_timing` is true. The new `time_eos_dxa` field is a subset of `time_eos`, not a separate item in `star_utils:total_times`; because it is updated inside the OpenMP cell loop, the printed label is `thread_time_eos_dxa`, with a companion `thread_time_eos_dxa/threads` estimate for comparison with wall-clock `time_eos`. The call counters also print `timing_num_get_eos_dxa_calls`, `timing_num_get_eos_dxa_skye_calls`, and the average Skye fraction of those calls. This is meant to distinguish “too many EOS calls” from “each composition-partial EOS call is too expensive”, especially across Skye and FreeEOS/Skye transition regions. The timing summary now also prints diagnostic thread-time counters for the dxa EOS path: component leaves (HELM, OPAL/SCVH, FreeEOS, PC, Skye, CMS, ideal), table lookup versus constrained X,Z expansion, common eosDT moment/blend/check work, and Skye’s ideal-ion, Coulomb, and packing composition-partial slices. These counters are diagnostics only; they are enabled by the existing star timing path and do not alter EOS values or solver assembly.

For FreeEOS and OPAL/SCVH table components, `get1_for_eosdt` now keeps the native table-coordinate pair

$$\left(\frac{\partial q}{\partial X}, \frac{\partial q}{\partial Z} \right)$$

compact until `set_table_dxa_from_XZ` expands it to isotope columns. This is the same constrained gauge as before,

$$D_H q = q_X - q_Z, \quad D_{He} q = -q_Z, \quad D_{metal} q = 0,$$

but the shared helper makes the FreeEOS/OPAL/SCVH basis conversion explicit and avoids repeating the mapping loops in each table-call path.

For the table-EOS and eosDT blend path, row-scoped requests now loop directly over `dxa_rows(:)` in the X/Z composition mapping and in `Do_Blend`, instead of scanning all `nv` rows and checking the row list inside the inner species-by-row loop. This is a performance cleanup only; the blend equation is unchanged:

$$\frac{\partial q}{\partial X_j} = \alpha \frac{\partial q_1}{\partial X_j} + \beta \frac{\partial q_2}{\partial X_j} + \frac{\partial \alpha}{\partial X_j} (q_1 - q_2).$$

The hot isotope-residual loop also avoids two unnecessary costs. First, the cross-species `d_sig_dxa` block is skipped entirely unless `implicit_diffusion_include_dsig_dxa` is true. Second, the structure partials from `sig_implicit_ad` are stored by directly mapping the same AD slots used by `star_utils:unpack_residual_partials`, avoiding one full residual-unpack call and full `nvar` scratch arrays for every isotope in every cell.

The nonlinear coefficient coupling is now separable: `implicit_diffusion_flag = .true.` keeps the fixed-coefficient diffusion solve implicit in composition, while `implicit_diffusion_include_dsig_structure = .true.` additionally adds the MLT/TDC structure derivative of `sigma`. The latter is off by default because near convective boundaries the MLT/TDC coefficient can switch or change by many orders of magnitude, so the nominally more complete Newton Jacobian can be less robust than a Picard update of the coefficient. This control is diagnostic and not a final fix. The production goal remains a stable nonlinear coefficient Jacobian for the implicit coefficient.

Implementation Log

Date	Change	Files	Verification
2026-05-17	Clarified the implicit-Dmix zone-selection policy. Solver iterations refresh the promoted coefficient value and derivatives from current <code>mlt_D_ad</code> , but the set of zones allowed into <code>Dmix_implicit</code> is held to the post-cleanup <code>mixing_type</code> from the last full <code>set_mixing_info</code> pass. A newly convective zone during Newton therefore waits until the next full mixing-info pass before joining the implicit component; this avoids changing which zones are mixed without the full MESA cleanup and boundary edits.	<code>star/private/implicit_Dmix_implicit</code> , <code>notes/eos_composition_parameters_implicit</code> , <code>notes/eos_composition_parameters_implicit</code>	Diffusion, convection, and static EOS implementation progress.md; New MESA implementation model run.
2026-05-17	Constrained solver-iteration implicit-Dmix promotion to the post-cleanup <code>mixing_type</code> from the last full <code>set_mixing_info</code> pass. The promoted coefficient value still comes from current <code>mlt_D_ad</code> , but zones converted by full-pass cleanup to minimum, overshoot, rotation, or no mixing no longer turn implicit transport back on inside Newton iterations. This targets the high-temperature retry pattern where large abundance corrections appeared in printed <code>mix type 7</code> regions.	<code>star/private/implicit_Dmix_implicit</code> , <code>notes/eos_composition_parameters_implicit</code> , <code>notes/eos_composition_parameters_implicit</code>	Diffusion, convection, and static EOS implementation progress.md; New MESA implementation model run.

Date	Change	Files	Verification
2026-05-17	Added a follow-up note on full-pass mixing cleanup versus solver-iteration implicit-Dmix refresh. Full <code>set_mixing_info</code> cleanup and pruning edits happen before the full <code>Dmix_implicit/Dmix_explicit</code> split, so they affect the stored components. The note records why allowing the solver-iteration zone selection to follow raw current <code>mlt_mixing_type</code> could re-open a pruned patchy convective cell; the subsequent row records the conservative correction.	<code>notes/eos_composition_parallelization_progress.md</code>	Static source review only. No MESA compile or model run.
2026-05-17	Cleaned up the implicit-diffusion sigma structure derivative path to use <code>star_utils:get_rho_face</code> for the face-density AD value, matching the existing hydro/turbulence helper instead of duplicating the same dq-weighted density expression locally.	<code>star/private/mix_info.f90</code> <code>notes/eos_composition_parallelization_progress.md</code>	Static source review only. No MESA compile or model run.

Date	Change	Files	Verification
2026-05-17	Clarified the implicit-Dmix component split after the solver-stability audit. <code>implicit_Dmix:set_Dmix_components</code> now has explicit full-pass and solver-iteration modes through <code>update_explicit_Dmix</code>. Full passes split the ordinary MESA total into non-negative <code>Dmix_implicit</code> and <code>Dmix_explicit</code> components and then rebuild total <code>D_mix</code>; solver iterations refresh only <code>Dmix_implicit</code> and keep <code>Dmix_explicit</code> fixed from the last full <code>set_mixing_info</code> pass. Code comments now describe when each mode is used, and notes now record that <code>D_mix</code> is the real total coefficient while the promoted-component derivatives are carried separately.	<code>star/private/implicit_Dmix.f90</code>; <code>star/private/hydro_vars.f90</code>; <code>star/private/mix_info.f90</code>; <code>star/private/solver_support.f90</code>; <code>notes/eos_composition_partials_implementation_progress.md</code>; <code>notes/eos_composition_partials_implementation_map_v2.md</code>	Static source and full-chain review only. No MESA compile or model runs.
2026-05-16	Audited ordinary EOS callers after the composition-partial work. Split burn and net one-zone burn still call legacy <code>eosDT_get</code>, which passes <code>include_composition_partials=.false.</code> and does not enter the Skye/FreeEOS composition-partial path. To avoid a shared-wrapper overhead, the legacy public EOS wrappers now use automatic full-row <code>d_dxa</code> scratch instead of heap allocating it on every call. This keeps the same returned legacy rows while reducing allocator cost for split burner, net, and EOS root-finder calls.	<code>eos/public/eos_lib.f90</code>; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static source audit only. No MESA compile or model runs.

Date	Change	Files	Verification
2026-05-16	Fixed solver-iteration implicit-Dmix consistency issues found during the high-temperature burning stability review. <code>implicit_Dmix:set_Dmix_components</code> now has explicit full-pass and solver-iteration modes. The solver-iteration mode refreshes <code>Dmix_implicit</code> while preserving <code>Dmix_explicit</code> from the last full <code>set_mixing_info</code> pass. The full pass builds the two components directly: promoted local MLT/TDC cells use <code>mlt_D_ad</code> in <code>Dmix_implicit</code>, and <code>Dmix_explicit</code> carries the non-implicit part of the ordinary MESA total. The 2026-05-17 correction restricts the promoted cells to post-cleanup full-pass <code>mixing_type</code>. <code>D_mix</code> is rebuilt as the component sum. <code>solver_support:set_vars_for_solver</code> now rebuilds diffusion sigmas during every implicit Newton iteration; the <code>d_sigma</code> controls only gate the optional coefficient-derivative Jacobian blocks.	<code>star/private/implicit_Dmix.f90</code>; <code>star/private/hydro_vars.f90</code>; <code>star/private/solver_support.f90</code>; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compile or model runs.
2026-05-16	Cached the selected Skye Coulomb OCP leaf values from the base liquid/solid branch evaluation and reused them in the hard-branch batched <code>dYA</code> composition partial path. This avoids recomputing the same per-species OCP free energies while leaving the transition and phase/latent paths on the full evaluation.	<code>eos/private/skye.f90</code>; <code>eos/private/skye_coulomb.f90</code>; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compile or model runs.

Date	Change	Files	Verification
2026-05-16	Reused the base Skye Coulomb AD result for the off-transition <code>dYe</code> and <code>dabar</code> Coulomb basis partials. In the hard liquid/solid branch, <code>dYe</code> enters only through <code>xnefer = avo*Ye*rho</code>, so <code>dF/dYe = (dF/drho)*rho/Ye</code>, and <code>dabar</code> enters only through the <code>kT/abar</code> conversion, so <code>dF/dabar = -F/abar</code>. The full companion calls remain for phase-transition cells and phase/latent rows.	<code>eos/private/skye.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compile or model run.
2026-05-15	Added a Skye Coulomb phase-branch hint for EOS composition partials. When the base EOS phase is exactly liquid or solid and phase/latent rows are not requested, the Coulomb <code>dx</code> path evaluates only the already-selected hard-min branch; cells near the phase transition still evaluate both branches. This targets the timing run where <code>thread_skye_dx_coul</code> dominated the remaining overhead.	<code>eos/private/skye.f90</code> ; <code>eos/private/skye_coulomb.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compile or model run.
2026-05-15	Fixed the Skye Coulomb batched <code>dYA</code> compile failure seen in <code>build.log</code>. The composition moments used by the batched number-fraction path are plain composition values; they are wrapped into the companion type only when seeding the liquid mixing-rule <code>d/dYA</code> derivative.	<code>eos/private/skye_coulomb.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Checked <code>build.log</code> failure <code>--check</code> and touched-file line scan clean. No MESA compile or model run.

Date	Change	Files	Verification
2026-05-15	Added the batched Skye Coulomb dYA path and star-side EOS composition-partial timing. The normal star lnPgas/lnE request now evaluates each OCP leaf once per active Skye species and reuses those values for all active number-fraction derivative columns; Ye, abar, and phase/latent requests still use the full companion path. Star timing now prints thread_time_eos_dxa, thread_time_eos_dxa/threads, timing_num_get_eos_dxa_calls, timing_num_get_eos_dxa_skye_calls, and the average Skye fraction for those calls. time_eos_dxa remains a subset of time_eos, so it is not included in star_utils:total_times.	eos/private/skye.f90; eos/private/skye_coulomb.f90; star/private/eos_support.f90; star/private/init.f90; star/job/run_star_support.f90; star_data/public/star_data_step_work.inc; notes/eos_composition_partials_implementation_progress.md	Static checks only. No MESA compile or model run.
2026-05-15	Added coarse EOS composition-partial timing counters so the 25x-per-call slowdown can be split by source. The diagnostic records thread-time and call counts for component leaves, table evaluation, table X,Z expansion, eosDT moment/blend/check overhead, and the Skye ideal/Coulomb/packing slices. It is enabled only through the existing star timing path.	eos/private/eos_timing.f90; eos/Makefile; eos/public/eos_lib.f90; eos/private/eosdt_eval.f90; eos/private/skye.f90; eos/private/ideal.f90; eos/private/eos_helm_eval.f90; eos/private/eospc_eval.f90; eos/private/eoscms_eval.f90; star/job/run_star_support.f90; star/private/init.f90; notes/eos_composition_partials_implementation_progress.md	Static checks only. No MESA compile or model run.

Date	Change	Files	Verification
2026-05-15	Removed unnecessary implicit-diffusion sigma-Jacobian storage work from the default path. <code>mix_info:get_convection_sigmas</code> now clears and updates <code>sig_implicit_ad(:)</code> only for <code>implicit_diffusion_include_dsig_structure</code>, and clears/scales <code>d_sig_dxa_m1/00(:, :)</code> only for the Ledoux <code>implicit_diffusion_include_dsig_dxa</code> block. <code>hydro_chem_eqns</code> now uses the same gates before copying sigma AD objects or entering the cross-species sigma loop. The later 2026-05-16 follow-up keeps the scalar <code>sig(:)</code> refresh during implicit Newton iterations, because that value is part of the current implicit residual, while keeping these optional derivative blocks gated.	<code>star/private/mix_info.f90</code> <code>star/private/hydro_chem_eqns.f90</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compile or model runs.
2026-05-15	Made the current performance refactor pass for EOS composition partials. <code>implicit_diffusion_flag</code> no longer forces hydro cell-centered EOS composition partials unless <code>implicit_diffusion_include_dsig_dxa</code> is also true. The implicit Brunt face EOS path now uses a moment-aware row-scoped wrapper. Skye ideal-ion composition partials now use one active-number-fraction derivative column at a time and cache the repeated $\log(n_j/n_{\{Q,j\}})$ AD term. FreeEOS and OPAL/SCVH table components now keep the native X,Z derivative pair compact until a shared helper expands it to the constrained isotope-column gauge.	<code>star/private/star_job_ctrls_io.f90</code> <code>star/private/ctrls_io.f90</code> <code>star/defaults/controls_dev_defaults</code> <code>star/defaults/star_job_dev_defaults</code> <code>star/private/eos_support.f90</code> <code>star/private/implicit_brunt.f90</code> <code>eos/private/skye_ideal.f90</code> <code>eos/private/eosdt_eval.f90</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compile or model runs.

Date	Change	Files	Verification
2026-05-15	Recorded the deferred row-sized EOS <code>dx</code> interface cleanup. Row-scoped calls already request only needed rows, but the internal component/blend interfaces still pass full <code>nv x species</code> derivative scratch arrays. A future cleanup can carry <code>nrows x species</code> blocks internally and scatter to legacy row indices at the public boundary. This is deferred until the derivative physics is settled.	<code>notes/eos_composition_partial_implementation.md</code>	No partial implementation. No compile or MESA model run.
2026-05-15	Added a design note for a future potential-level FreeEOS/Skye blend. If FreeEOS can expose a consistent Helmholtz free energy and the required ρ, T, X derivatives through <code>eosDT</code> , the transition could blend f first and derive P, E, S, χ , and composition rows from the single blended potential. This is recorded as a separate long-term thermodynamic endpoint, not a current implementation change.	<code>notes/eos_composition_partial_implementation.md</code>	No partial implementation. No compile or MESA model run.

Date	Change	Files	Verification
2026-05-15	Reused star-side composition moments on the production EOS <code>dx</code> path. <code>micro:do_eos_for_cell</code> now calls the moment-aware row-scoped wrapper, <code>Get_eosDT_Results</code> carries one set of raw moment derivative columns through eosDT only when the composition-partial path is requested, and Skye, ideal, HELM, and blend-alpha chain rules can reuse those columns instead of repeating <code>basic_composition_info</code>. Removed the explicit heap allocation from <code>combine_for_eosdt</code> blend scratch work, and completed the PC Gamma-limit log-rho blend <code>dalpha/dabar,dalpha/dzbar</code> branch.	<code>eos/public/eos_lib.f90</code>; <code>eos/private/eosdt_eval.f90</code>; <code>eos/private/eospc_eval.f90</code>; <code>eos/private/ideal.f90</code>; <code>eos/private/skye.f90</code>; <code>eos/private/eos_helm_eval.f90</code>; <code>star/private/eos_support.f90</code>; <code>star/private/micro.f90</code>; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compile or model run.
2026-05-15	Audited the Skye composition-partial NaN path. The companion math has the expected singular points (<code>log</code>, <code>sqrt</code>, divisions), but for valid Skye states the active number fractions and thermodynamic denominators should be positive. The remaining concrete contamination risk was the basis assembly evaluating exact zero coefficients as <code>0*bad_basis</code>. <code>skye:get</code> now skips exact zero terms in the Coulomb/electron basis expansion and returns <code>ierr</code> if any requested Skye <code>d_dxa</code> row is non-finite before eosDT blending; the eosDT wrapper returns immediately on that error instead of stamping Skye fraction fields after a failed call.	<code>eos/private/skye.f90</code>; <code>notes/eos_composition_partials_implementation_progress.md</code>	<code>git diff --check</code>; <code>For its implementation</code> No MESA compile or model run.

Date	Change	Files	Verification
2026-05-15	Fixed a table-EOS raw X/Z mapping NaN path exposed by hydro-only EOS composition partials. OPAL/SCVH and FreeEOS now assign the H and He columns explicitly and leave metal columns at zero instead of evaluating $0 \cdot d_X + 0 \cdot d_Z$, which can turn an unused NaN table-coordinate derivative into NaNs for every species column. Added a component-agnostic star-side guard so non-finite $\ln P_{\text{gas}}$ or $\ln E$ composition rows from any EOS component or blend, including Skye, are reported at <code>store_eos_for_cell</code> with EOS fractions and cell context instead of propagating into <code>dv_dt/dlnE_dt</code> matrix entries.	<code>eos/private/eosdt_eval.f90</code> ; <code>star/private/micro.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compile or model run.
2026-05-15	Refactored Skye Coulomb composition partials from per-isotope companion evaluations to basis derivatives in <code>Ye</code> , <code>abar</code> , and active Skye number fractions. Each network isotope column is assembled by linear combination, reducing the expensive Coulomb companion calls from <code>species</code> to <code>relevant_species + 2</code> per cell. Also recorded that <code>mass_fraction_limit_for_Skye</code> only limits the active Coulomb species set; hydro-solve EOS <code>dx</code> columns are not zeroed by a mass-fraction cutoff.	<code>eos/private/skye.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	<code>git diff --check;</code> Static checks only. No MESA compile or model run.

Date	Change	Files	Verification
2026-05-15	Added an in-code note in the <code>star_solver</code> diagnostic branch: the production star EOS call requests composition partials only for <code>lnPgas</code> and <code>lnE</code> , so <code>grad_ad</code> abundance <code>solver_test_partials</code> needs a full EOS <code>dx</code> row request. Values and structure derivatives for <code>grad_ad</code> are still returned by the normal EOS call.	<code>star/private/star_solver.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compilation or run.
2026-05-15	Tightened the EOS composition-partial hot path without changing the physics. Skye now only computes companion phase and latent-heat derivatives when those rows are requested, so the usual star <code>lnPgas/lnE</code> request pays only for the free-energy composition derivative. The table-EOS X/Z mapping and <code>Do_Blend</code> now loop over the requested <code>dx_rows(:)</code> directly instead of scanning every EOS row inside the species loop.	<code>eos/private/skye.f90</code> ; <code>eos/private/skye_coulomb.f90</code> ; <code>eos/private/eosdt_eval.f90</code> ; <code>eos/private/eosdt_support.f90</code> ; <code>eos/private/skye_thermodynamics.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	<code>git diff --check</code> ; hed-file line scan clean. No MESA compile or run.
2026-05-15	Added dev <code>star_job</code> control <code>implicit_diffusion_include</code> default false. This separates the same-species implicit composition solve from the nonlinear MLT/TDC coefficient Jacobian. The scalar <code>sig(:)</code> values still refresh from the current <code>Dmix_implicit</code> value during implicit Newton iterations; this control gates only the structure partials from <code>sig_implicit_ad</code> and their residual-Jacobian insertion.	<code>star/defaults/star_job_defaults.f90</code> ; <code>star_data/private/star_job_controls.f90</code> ; <code>star/private/star_job_controls.f90</code> ; <code>star/private/mix_info.f90</code> ; <code>star/private/hydro_chem_eqns.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA compilation or run.

Date	Change	Files	Verification
2026-05-15	Forced the <code>d_sig_dxa_m1/00</code> path off when <code>use_Ledoux_criterion</code> is false, on both the producer side in <code>mix_info</code> and the isotope-Jacobian consumer side in <code>hydro_chem_eqns</code>. In Schwarzschild mode the Ledoux composition term is not part of the MLT/TDC coefficient, so the implicit diffusion Jacobian should not consume Brunt composition derivatives even if <code>implicit_diffusion_include_dsig_dxa</code> was set true. This also avoids propagating stale or NaN Brunt composition derivatives into the abundance Jacobian in non-Ledoux runs.	<code>star/private/mix_info.f90</code> <code>star/private/hydro_chem_eqns.f90</code> <code>star/defaults/star_job_defaults</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	Static review only. No HSA compile or model runs.
2026-05-15	Changed EOS composition partials through blended EOS regions to hold the blend weights fixed with respect to composition. <code>eosdt_support:Do_Blend</code> still includes the blend-weight derivatives for <code>logRho</code> and <code>logT</code>, but no longer adds the artificial <code>dalpha/dxa*(res_1-res_2)</code> term to composition rows. This targets stiff or bad composition derivatives at numerical EOS selectors such as the FreeEOS-to-Skye transition. <code>Get_eosDT_Results</code> now rejects bad requested composition partial rows before they can enter the star solver Jacobian.	<code>eos/private/eosdt_support.f90</code> <code>eos/private/eosdt_eval.f90</code> <code>docs/source/eos/overview.md</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	Static review only. No HSA compile or model runs.

Date	Change	Files	Verification
2026-05-15	Removed two hot isotope-residual costs in the implicit diffusion Jacobian. The <code>d_sig_dxa</code> cross-species loop is now skipped in <code>hydro_chem_eqns</code> unless <code>implicit_diffusion_include_dsig_dxa</code> is true, matching the producer-side control in <code>mix_info</code> . The <code>sig_implicit_ad</code> structure partials are now stored directly from the AD slot array rather than by calling the generic <code>save_eqn_residual_info</code> unpacker once per isotope. This leaves the diffusion equations unchanged: only the Jacobian assembly path is cheaper.	<code>star/private/hydro_chem_eqns.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA implementation run.
2026-05-15	Removed a per-call Skye heap allocation in the EOS composition-partial path. <code>skye:get</code> now asks <code>get_active_number_fraction_partial</code> for the single active-number-fraction derivative column used by the current species loop, instead of allocating and filling the full <code>relevant_species</code> by <code>species</code> matrix.	<code>eos/private/skye.f90</code> ; <code>eos/private/eos_composition_partials.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA implementation run.
2026-05-15	Added dev <code>star_job</code> control <code>implicit_diffusion_use_brunt_gauss</code> default false. The implicit Ledoux Brunt path now defaults to the original one-face-EOS-partial contraction and only uses the two-point Gauss composition path when this control is true. Also delayed eosDT blend scratch allocation until after pure component cases can return, reducing heap work for non-blending EOS states.	<code>star/defaults/star_job_defaults.f90</code> ; <code>star_data/private/star_job_defaults.f90</code> ; <code>star/private/implicit_brunt.f90</code> ; <code>eos/private/eosdt_eval.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Static checks only. No MESA implementation run.

Date	Change	Files	Verification
2026-05-15	Added row-scoped cheaper EOS composition wrappers and switched production star/Brunt callers onto them. <code>eosDT_get_dxa_rows</code> passes a requested <code>dxa_rows(:)</code> list through <code>eosDT</code> and <code>Do_Blend</code> ; <code>get_eos_star_dxa</code> requests only <code>lnPgas</code> and <code>lnE</code> ; <code>get_eos_brunt_dxa</code> requests only <code>lnPgas</code> , adding <code>chiT</code> and <code>chiRho</code> only for the optional <code>d_sigma/dX</code> block. Added MESA-style comments above the cheaper-wrapper API calls. Also added a two-point Gauss path-average formulation for the implicit Brunt numerator; the follow-up row above made it optional and default-off. The earlier FreeEOS/OPAL raw-gauge fix remains in <code>get1_for_eosdt</code> .	<code>eos/public/eos_lib.f90</code> ; <code>git diff --check</code> ; <code>eos/private/eosdt_eval.f90</code> <code>eos/private/eosdt_support.f90</code> <code>eos/private/eos_composition_partials.f90</code> ; edit. <code>eos/private/skye_thermodynamics.f90</code> ; <code>eos/private/skye.f90</code> ; <code>eos/private/ideal.f90</code> ; <code>eos/private/eos_helm_eval.f90</code> ; <code>eos/private/eoscms_eval.f90</code> ; <code>eos/private/eospc_eval.f90</code> ; <code>star/private/eos_support.f90</code> ; <code>star/private/micro.f90</code> ; <code>star/private/implicit_brunt.f90</code> ; <code>notes/eos_composition_partials_implementation_progress.md</code>	Untracked-file line scan clean. No MESA compile or model run.
2026-05-15	Recorded the next implicit-diffusion diagnosis: disabling <code>d_sig_dxa_m1/00</code> did not remove the main-sequence <code>equ_he4</code> failure, so the remaining EOS-composition path specific to implicit diffusion was the implicit Brunt pressure-composition numerator. For FreeEOS/OPAL table regions this was then a one-point face derivative of <code>lnPgas</code> ; the follow-up edit above replaced it with a path-averaged analytic derivative. A secant diagnostic at fixed face <code>T,rho</code> remains a possible debugging tool.	<code>notes/eos_composition_partials_implementation_progress.md</code>	No MESA compile or model run.

Date	Change	Files	Verification
2026-05-15	Corrected the OPAL/SCVH and FreeEOS table-EOS isotope gauge for the star solver. The previous normalized-coordinate mapping was equivalent for constrained finite differences and for the Brunt contraction, but it changed the raw row-constant gauge seen by energy/pressure equation partials. get1_for_eosdt now maps table derivatives through basic_composition_info coordinates: hydrogen gives $q_X - q_Z$, helium gives $-q_Z$, and explicit metals give zero raw table-coordinate derivative. Sink-projected checks still give $D_{\text{he4}}^{\text{h1}} q = -q_X$ and $D_{\text{metal}}^{\text{h1}} q = q_Z - q_X$.	<code>eos/private/eosdt_eval.f90</code> <code>eos/plotter/composition_partials/README.md</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	Static review only. No controls/HSA model
2026-05-15	Added a dev star_job switch implicit_diffusion_include default false, for the high-gain cross-species composition block in the implicit diffusion Jacobian. mix_info:get_convection_sigmas only fills d_sig_dxa_m1/00 when this switch is true. implicit_brunt:get_implicit_brunt_B also skips the d_brunt_B_dxa_m1/00 derivative loop when those terms will not be used. This is targeted at the main-sequence pattern where equ_he4 residuals drive large neighboring o16 corrections through the convective-boundary d_sigma/dX block. The later 2026-05-16 update keeps scalar sigma values current during implicit solver iterations but leaves this cross-species Jacobian block gated by the control.	<code>star/defaults/star_job_defaults.f90</code> <code>star_data/private/star_job_controls/HSA_model</code> <code>star/private/star_job_controls/io.f90</code> <code>star/private/mix_info.f90</code> <code>star/private/implicit_brunt.f90</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	Static review only. No controls/HSA model

Date	Change	Files	Verification
2026-05-15	Reduced overhead in the implicit Brunt hot path. <code>hydro_vars:set_hydro_vars</code> now only calls the face-EOS implicit Brunt path when both <code>implicit_diffusion_flag</code> and <code>use_Ledoux_criterion</code> are true; non-Ledoux implicit runs explicitly zero the AD composition term without making face EOS calls. <code>implicit_brunt:do_implicit_brunt_B</code> now allocates <code>xa_face</code> , <code>chiX_face</code> , and <code>d_eos_dxa</code> once per OpenMP worker instead of once per face.	<code>star/private/hydro_vars.f90</code> <code>star/private/implicit_brunt.f90</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	f90; or MESA model partial implementation progress.md
2026-05-15	Added the first implicit-Dmix solver refresh. <code>hydro_vars:set_hydro_vars</code> calls <code>set_Dmix_components</code> after the current MLT/TDC <code>set_mlt_vars</code> pass when <code>set_mixing_info</code> is skipped, and <code>solver_support:set_vars_for_solver</code> refreshes diffusion sigmas during implicit solver iterations. Later follow-ups keep <code>Dmix_explicit</code> fixed in this path and restrict the implicit-zone selection to the post-cleanup full-pass <code>mixing_type</code> .	<code>star/private/hydro_vars.f90</code> <code>star/private/solver_support.f90</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	f90; or MESA model partial implementation progress.md
2026-05-14	Created progress copy from implementation map.	<code>notes/eos_composition_partials_implementation_progress.md</code>	Reader-implicit and f90; or MESA model
2026-05-14	Added first <code>Dmix_implicit</code> infrastructure slice on branch <code>EbF/implicit_diffusion</code> ; kept the user-facing flag dev-only.	<code>star/private/implicit_Dmix.f90</code> <code>star/private/mix_info.f90</code> <code>star/private/star_job_ctrls_io.f90</code> <code>star/defaults/star_job_dev.defaults</code> <code>star_data/private/star_job_controls_dev.inc</code> <code>star_data/public/star_data_step_work.inc</code> <code>star/private/alloc.f90</code> <code>star/private/read_model.f90</code> <code>star/private/adjust_mesh_split_merge.f90</code> <code>star/Makefile</code>	diff --check only. No MESA compile/run.
2026-05-14	Added private EOS composition-moment derivative helper.	<code>eos/private/eos_composition_moment.f90</code> <code>eos/Makefile</code>	implicit and line-length scan only. No MESA compile/run.

Date	Change	Files	Verification
2026-05-14	Added dev include_eos_composition_partial_data &controls flag and auto-enabled it from implicit_diffusion_flag. Clarified that false means current EOS behavior.	star/defaults/controls_dev_implicit_diffusion_flag.f90; star_data/private/star_controls_dev_implicit_diffusion_flag.f90; star/private/ctrls_io.f90; star/private/star_job_ctrls_io.f90	git diff --check only. No MESA compile/run.
2026-05-14	Renamed the dev star_job Dmix switch to implicit_diffusion_flag; kept Dmix_implicit as the implicit coefficient name.	star/defaults/star_job_dev_implicit_diffusion_flag.f90; star_data/private/star_job_dev_implicit_diffusion_flag.f90; star/private/star_job_ctrls_io.f90; star/private/ctrls_io.f90; star/private/implicit_Dmix.f90	git diff --check only. No MESA compile/run.
2026-05-14	Added internal full-d_dxa EOS call path for the dev composition-partial flag. Under the dev flag, s% d_eos_dxa now stores all EOS result rows; otherwise it keeps legacy two-row storage. Added a defensive do_eos expansion if the flag is enabled after allocation. Skipped the finite-difference EOS-partial fallback on the new path.	eos/public/eos_lib.f90; git diff --check only. star/private/eos_support.f90; No MESA compile/run. star/private/micro.f90; star/private/hydro_eqns.f90; star/private/alloc.f90; star/private/adjust_xyz.f90; star_data/public/star_data_step_work.inc	
2026-05-14	Added full-row ideal-ion composition partial packing for Skye and ideal, plus Skye HELM electron Ye chain-rule contribution. Added ion-offset composition derivative helper.	eos/private/skye.f90; git diff --check and eos/private/ideal.f90; line-length scan only. No eos/private/skye_ideal.f90; MESA compile/run. eos/private/skye_thermodynamics.f90; eos/private/ion_offset.f90	
2026-05-14	Added eosDT blend plumbing for d_alfa_dxa(:) with the correct dalpha/dxa*(res_1-res_2) term; component alpha derivatives still pass zero.	eos/private/eosdt_eval.f90; git diff --check and eos/private/eosdt_support.f90; line-length scan only. No MESA compile/run.	
2026-05-15	Wired the first nonzero eosDT blend-alpha composition derivatives: PC Gamma-limit blends now map dalpha/dabar and dalpha/dzbar through raw composition moment partials, and OPAL/SCVH high-Z HELM-reduction blends now map dalpha/dZ by isotope charge.	eos/private/eosdt_eval.f90; git diff --check and eos/private/eospc_eval.f90; line-length scan only. No MESA compile/run.	

Date	Change	Files	Verification
2026-05-15	Added composition derivatives for the full Skye eosDT polygon alpha by differentiating the moving <code>abar,zbar</code> polygon boundaries and mapping <code>dalpha/dabar,dalpha/dzbar</code> through the EOS moment helper.	<code>eos/private/skye.f90</code> ; <code>eos/private/eosdt_eval.f90</code>	<code>git diff --check</code> and touched-file line scan only. No MESA compile/run.
2026-05-15	Downloaded the Skye arXiv source locally and completed the third-order HELM electron <code>F_dye</code> chain-rule object, including the fourth density basis derivative needed for the <code>rho,rho,rho</code> slot.	<code>notes/skye_arxiv_2104.00691v4</code> ; <code>notes/skye_arxiv_2104.00691v4</code> ; <code>eos/private/helm_polynomial.f90</code> ; <code>eos/private/skye_ideal.f90</code>	<code>git diff --check</code> ; touched-file line scan still reports only existing long lines in <code>skye_ideal.f90</code> . No MESA compile/run.
2026-05-15	Moved the active-set number-fraction Jacobian into the shared EOS composition helper so Skye Coulomb partials can use the same arbitrary-network derivative source as the ideal-ion term.	<code>eos/private/eos_composition.f90</code> ; <code>eos/private/skye_ideal.f90</code>	<code>git diff --check</code> ; touched-file line scan still reports only existing long lines in <code>skye_ideal.f90</code> . No MESA compile/run.
2026-05-15	Added the private Skye composition companion-AD module. It carries a normal <code>auto_diff_real_2var_order3</code> value and a companion <code>d/dX_j</code> object through arithmetic, elementary functions, min/max branches, powers, and <code>differentiate_1/2</code> .	<code>eos/private/skye_composition.f90</code> ; <code>eos/Makefile</code>	<code>git diff --check</code> and touched-file line scan. No MESA compile/run.
2026-05-15	Added companion versions of the Skye Coulomb leaf formulas for liquid and solid OCP free energies, liquid and solid screening corrections, liquid and solid mixing corrections, and solid two-component mixing fits. Added integer-literal arithmetic overloads to the companion type so copied Skye algebra can stay close to the original formulas.	<code>eos/private/skye_composition.f90</code> ; <code>eos/private/skye_coulomb_liquid.f90</code> ; <code>eos/private/skye_coulomb_solid.f90</code>	<code>git diff --check</code> ; line scan still reports only existing long lines in <code>skye_coulomb_liquid.f90</code> and nearby legacy comments/signatures. No MESA compile/run.

Date	Change	Files	Verification
2026-05-15	Wired the first full Skye Coulomb composition derivative path. nonideal_corrections_dxa now propagates companion derivatives through r_s , Gamma_e , EXCOR7 , mixing entropy, OCP sums, Gamma-limit extrapolation, unit conversion by abar , and the phase softmin. skye_eos now calls it for each original isotope and adds F_coul_dxa , phase_dxa , and latent derivatives to the full composition packing path.	eos/private/skye.f90 ; eos/private/skye_coulomb.f90 ; eos/private/skye_coulomb_skye.f90 ; eos/private/skye_coulomb_legacy.f90 ; eos/private/skye_composition.f90	git diff --check ; line skye.f90 ; reports existing long lines. skye.f90 and nearby signatures. No MESA compile/run.
2026-05-15	Threaded the internal include_composition_partials request through Get_eosDT_Results and the eosDT component interface. Legacy eosDT_get passes false and the new eosDT_get_full_dxa path passes true, so Skye companion derivatives, ideal full-row composition packing, and eosDT blend-alpha composition terms are paid for only on the dev full-partials path.	eos/public/eos_lib.f90 ; eos/private/eosdt_eval.f90 ; eos/private/skye.f90 ; eos/private/ideal.f90 ; eos/private/eoscms_eval.f90 ; eos/private/eos_helm_eval.f90 ; eos/private/eospc_eval.f90	git diff --check ; static all-chain scan; touched-file line scan still reports existing long lines. skye.f90 ; MESA compile/run.
2026-05-15	Tightened the Skye/ideal value paths so the shared composition-moment helper is only called when the internal full-composition-partial request is true. In Skye, the helper uses a scratch Ye so the derivative path cannot perturb the value-path electron fraction.	eos/private/skye.f90 ; eos/private/ideal.f90	git diff --check ; regenerated progress PDF. No MESA compile/run.
2026-05-15	Zeroed the CMS inactive phase and EOS-fraction d_dxa rows after the radiation add, matching the existing value, density, and temperature derivative zeroing in that wrapper.	eos/private/eoscms_eval.f90	git diff --check ; regenerated progress PDF. No MESA compile/run.

Date	Change	Files	Verification
2026-05-15	Audited the non-Skye component wrappers for the full-d_dxa path. HELM already returns analytic composition rows through its <code>abar, zbar</code> derivatives, OPAL/SCVH and FreeEOS table wrappers already zero inactive phase/fraction rows, CMS now does the same, and pure PC remains an explicit zero-derivative component until a PC-local analytic composition path is added.	<code>eos/private/eos_helm_eval.f90</code> ; <code>eos/private/eosdt_eval.f90</code> ; <code>eos/private/eoscms_eval.f90</code> ; <code>eos/private/eospc_eval.f90</code>	<code>git diff --check</code> ; source audit only. No MESA compile/run.
2026-05-15	Moved the full-d_eos_dxa allocation expansion in <code>micro</code> into a shared helper and called it from both <code>do_eos</code> and <code>set_eos_with_mask</code> , so masked EOS refreshes used by convective premixing and phase separation see the same dev full-row storage guarantee.	<code>star/private/micro.f90</code>	<code>git diff --check</code> ; regenerated progress PDF. No MESA compile/run.
2026-05-15	Recorded two implementation decisions: pure-PC internal composition partials are deferred because they need a PC-local derivative path through MELANGE9 and PC mixing/phase algebra; the current Brunt Jacobian intentionally treats the face EOS composition coefficient as a current-iterate coefficient, so second EOS composition derivatives are not part of this slice. Also recorded the transport policy: MLT, TDC, and semiconvection are implicit components; rotation and other non-semiconvective transport stay semi-implicit/explicit unless separately promoted; <code>turb_support</code> updates per iteration.	<code>notes/eos_composition_partials/Implementation</code>	<code>git diff --check</code> ; regenerated progress PDF. No MESA compile/run.

Date	Change	Files	Verification
2026-05-14	Added implicit Brunt and scalar-AD mixing plumbing. <code>implicit_brunt</code> computes the face complete EOS-composition Brunt term from full EOS composition partials, stores AD Brunt/gradL-composition terms, and is called during solver iterations under <code>implicit_diffusion_flag</code>. MLT/TDC now receive a scalar AD <code>gradL_composition_term</code>; <code>Dmix_implicit</code> is the AD implicit coefficient and <code>Dmix_explicit</code> is the semi-implicit explicit coefficient. Follow-up edit made <code>brunt_B_ad</code> carry structure derivatives through the pressure denominator and face <code>chi</code> terms.	<code>star/private/implicit_brunt.f90</code> <code>star/private/hydro_vars.f90</code> <code>star/private/turb_info.f90</code> <code>star/private/turb_support.f90</code> <code>star/private/implicit_Dmix.f90</code> <code>star_data/public/star_data_step_work.inc</code> <code>star/private/alloc.f90</code> <code>star/private/read_model.f90</code> <code>star/private/adjust_mesh_split_merge.f90</code> <code>star/public/star_lib.f90</code> <code>star/Makefile</code> <code>star/defaults/star_job_dev.defaults</code>	<code>brunt f90f --check</code> and line-length scan only. No MESA compile/run.
2026-05-14	Added the first tridiagonal Jacobian coupling from implicit Brunt into the isotope equations. The implicit Brunt path now fills species-indexed <code>d_brunt_B_dxa_m1/00</code>. <code>mix_info:get_convection_sigmas</code> maps them into <code>d_sig_dxa_m1/00</code> through <code>dDmix_implicit/dB</code> and stores <code>sig_implicit_ad(:)</code> for structure partials of the implicit sigma. <code>hydro_chem_eqns:do1_chem_eqns</code> adds the structure contribution with <code>shift_p1</code> on the upper face and adds the cross-species composition terms to the existing <code>dxdt_mix</code> residual. The implicit Brunt path deliberately does not smooth <code>brunt_B</code> or its composition derivatives.	<code>star/private/implicit_brunt.f90</code> <code>star/private/mix_info.f90</code> <code>star/private/hydro_chem_eqns.f90</code> <code>star_data/public/star_data_step_work.inc</code> <code>star/private/alloc.f90</code> <code>star/private/read_model.f90</code> <code>star/private/adjust_mesh_split_merge.f90</code>	<code>brunt f90f --check</code> and line-length scan only. No MESA compile/run.
2026-05-15	Added the direct mass-correction derivative to the implicit Brunt species chain rule.	<code>star/private/implicit_brunt.f90</code>	<code>brunt f90f --check</code> and line-length scan only. No MESA compile/run.

Date	Change	Files	Verification
2026-05-15	Cleaned up the implicit Brunt implementation names and comments so the code reads as the complete EOS-composition form: q_face became chiX_face , the composition-contraction derivative is dcomp_* , and denominator/temperature/mass-correction derivatives are named explicitly.	star/private/implicit_brunt.f90 notes/eos_composition_partials_implicit.f90	git diff --check; git diff --check; No MESA compile/run.
2026-05-15	Used the first approved compile cycle. ./clean completed; ./install stopped in the EOS build because pack_composition_partials selected %val directly from differentiate_1/2(...) function results. Patched the source by storing those derivative results in local AD temporaries first.	eos/private/skye_thermodynamics.f90 notes/eos_composition_partials_implicit.f90	git diff --check; git diff --check; clean. Compile failed before patch.
2026-05-15	Fixed star-side AD import visibility for the new implicit diffusion modules/subroutines. The AD assignment and arithmetic generics now follow the existing MESA auto_diff_support pattern.	star/private/implicit_Dmix.f90 star/private/implicit_brunt.f90 star/private/mix_info.f90 star/private/hydro_chem_eqns.f90	./install passes through EOS and star.
2026-05-15	Tightened the implicit transport component policy. Dmix_implicit now accepts local mlt_D_ad only for convective_mixing and semiconvective_mixing , so MLT/TDC convective regions and semiconvection are implicit while thermohaline and other post-processing remain in Dmix_explicit .	star/private/implicit_Dmix.f90 notes/eos_composition_partials_implicit.f90	git diff --check; git diff --check; Final ./install now succeeds after the local atm golden update.

Date	Change	Files	Verification
2026-05-15	Reduced default-path overhead from the implicit diffusion scaffolding. <code>get_convection_sigmas</code> now touches the species-sized implicit sigma derivative arrays only when <code>implicit_diffusion_flag</code> is true, and <code>set_mixing_info</code> only calls <code>set_Dmix_components</code> on the dev implicit path.	<code>star/private/mix_info.f90</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	<code>git diff --check;</code> Final <code>./install</code> now succeeds after the local atm golden update.
2026-05-15	Updated the local atm golden output for the one-last-digit tau change produced by the current build, preserving the expected trailing blank line.	<code>atm/test/test_output</code>	<code>git diff --check;</code> <code>./install</code> successful.
2026-05-15	Guarded the shared Brunt smoothing step so <code>implicit_diffusion_flag</code> keeps the implicit Brunt value unsmoothed all the way through <code>set_grads</code> .	<code>star/private/hydro_vars.f90</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	<code>git diff --check;</code> <code>./install</code> successful.
2026-05-15	Added a dev control check that rejects <code>implicit_diffusion_flag</code> with <code>use_other_brunt</code> , <code>use_other_brunt_smoothing</code> , or <code>use_other_mlt_results</code> , because the implicit path needs the internal EOS-composition Brunt derivative, deliberately leaves Brunt unsmoothed, and needs the built-in MLT/TDC AD path for <code>dDmix/dB</code> .	<code>star/private/ctrls_io.f90</code> <code>star/defaults/star_job_dev_defaults</code> <code>notes/eos_composition_partials_implementation_progress.md</code>	<code>git diff --check;</code> <code>./install</code> successful.

Date	Change	Files	Verification
2026-05-15	Added a local EOS composition-partial validation plotter. The driver compares <code>eosDT_get_full_dxa</code> against constrained Ridders finite differences using <code>he4</code> as the sink species, writes a 1D validation CSV, and writes a 2D default-EOS-domain contour CSV. The plotting script now uses matplotlib with the MESA EOS-region style and emits contour-only PDF/PNG figures plus a combined multi-page report.	<code>eos/plotter/composition_partial/MakeFile;</code> <code>eos/plotter/composition_partial/README.md</code> <code>eos/plotter/composition_partial/sinks/eos_comp</code> <code>eos/plotter/composition_partial/plot_composition_partials.py</code>	Partial/MakeFile; Partial/README.md Partial/sinks/eos_comp Partial/plot_composition_partials.py in the plotter directory.
2026-05-15	Fixed the HELM <code>mu</code> composition derivative. HELM returns <code>mu=abar/(1+zbar)</code> , so the analytic component slots are <code>dmu/dabar=1/(1+zbar)</code> and <code>dmu/dzbar=-mu/(1+zbar)</code> .	<code>eos/private/eos_helm_evalFreeEOS ./install</code>	FreeEOS ./install successful; regenerated plotter output shows <code>mu</code> max relative error below <code>6e-9</code> on the cool HELM/FreeEOS sweep.
2026-05-15	Recorded the first numeric EOS validation result. The default-control contour plotter covers the same <code>(logRho, logT)</code> domain as the default EOS plotter, with a denser <code>150 x 150</code> grid. Rows directly used by implicit Brunt are good in the Skye-dominated finite-difference sweeps at the current tolerance, while the cool HELM/FreeEOS path still has large discrepancies in derived rows such as <code>chiRho</code> , <code>grad_ad</code> , <code>gamma1</code> , and <code>Cp</code> . PC and CMS still need dedicated forced-component/state grids.	<code>eos/plotter/composition_partial/CSV/eos_composition_partial-2s/data/eos_composition_partial-2s/figure150.pdf;</code> <code>eos/plotter/composition_partial/figure150.png;</code> <code>eos/plotter/composition_partial/figure150.png;</code> <code>notes/eos_composition_partial/implementationprogress.md</code>	Partial/CSV/eos_composition_partial-2s/data/eos_composition_partial-2s/figure150.pdf; Partial/figure150.png; Partial/figure150.png; notes/eos_composition_partial/implementationprogress.md make implementationprogress.md points for the current solar-style grid.

Date	Change	Files	Verification
2026-05-15	First pass at the OPAL/SCVH and FreeEOS X/Z table composition-partial gauge mapped native table derivatives dq/dX and dq/dZ through normalized composition coordinates. This preserved constrained sink-projected derivatives and the Brunt contraction, but a later correction switched the raw star-solver gauge to match basic_composition_info .	<code>eos/private/eosdt_eval.f90</code> , <code>eos/plotter/composition_partials/README.md</code> , <code>notes/eos_composition_partials_implementation_progress.md</code>	install successful; made as README made partial implementation plotter directory for that intermediate gauge.
2026-05-15	Added DFRIDR contour validation pages for constrained composition partials of P_{gas} , μ , and $\ln E$. These pages compare he4 , c12 , and o16 directions against a fixed non-plotted h1 sink, so the sink species is not the value being plotted. No-EOS coverage cells are masked white and are excluded from colorbar scaling.	<code>eos/plotter/composition_partials/README.md</code> , <code>eos/plotter/composition_partials/plot_composition_partials.py</code> , <code>eos/plotter/composition_partials/README.md</code> , <code>notes/eos_composition_partials_implementation_progress.md</code>	figures/dfridr_relerr_Pgas.{pdf,png}, figures/dfridr_relerr_mu.{pdf,png}, figures/dfridr_relerr_lnE.{pdf,png}, and the combined contour report.

Current Implicit Brunt And Dmix Math

Derivative level needed.

The dev path needs first composition partials of EOS result rows, not just the two legacy rows. In code, **star_solver.f90** already consumes $\partial \ln E / \partial X_j$, $\partial \ln P_{\text{gas}} / \partial X_j$, and $\partial \nabla_{\text{ad}} / \partial X_j$ through the sink-species basis. The new implicit Brunt path additionally consumes $\partial \ln P_{\text{gas}} / \partial X_j$, $\partial \chi_T / \partial X_j$, and $\partial \chi_\rho / \partial X_j$.

For Skye this means each species companion free energy

$$F_{x_j} = \left. \frac{\partial F}{\partial X_j} \right|_{\rho, T}$$

must carry its own T, ρ derivatives through third order, because **pack_composition_partials** differentiates F_{x_j} to form rows such as $dE/d\rho$, χ_T , χ_ρ , Γ_1 , and ∇_{ad} . These are derived chain-rule objects; filling the result rows without deriving the corresponding F_{x_j} derivatives is not equivalent.

For the HELM electron Y_e companion, these third-order slots are therefore required. Leaving them as zero would incorrectly zero parts of the full composition rows for quantities derived by differentiating F_{x_j} , including χ_T , χ_ρ , Γ_1 , Γ_3 , ∇_{ad} , and related energy/entropy derivative rows. The branch now fills the needed **F_dye** third-order slots analytically in `eos/private/skye_ideal.f90`; the ρ, ρ, ρ slot uses the new fourth density derivatives of the HELM Hermite basis in `eos/private/helm_polynomials.f90`. Remaining zero composition rows are intentional or deferred component paths, not the HELM electron derivative slots needed by the Skye full-row pack.

New dev-path face Brunt value.

For a face between cells k and $k-1$, the new implicit branch forms a face composition

$$x_{j,f} = \alpha x_{j,k} + (1 - \alpha) x_{j,k-1}, \quad \alpha = \frac{dq_{k-1}}{dq_{k-1} + dq_k}.$$

By default, it evaluates the pressure-composition numerator from the EOS partial at the face composition:

$$\chi_{X_j,f} = \left. \frac{\partial \ln P}{\partial x_j} \right|_{\rho_f, T_f, x_f} = \frac{P_{\text{gas},f}}{P_f} \left. \frac{\partial \ln P_{\text{gas}}}{\partial x_j} \right|_{\rho_f, T_f, x_f}.$$

If `implicit_diffusion_use_brunt_gauss_path` is true, the code instead evaluates a two-point Gauss path average through the constrained composition jump,

$$x_j(\lambda) = (1 - \lambda)x_{j,k-1} + \lambda x_{j,k}, \quad \lambda_{\pm} = \frac{1}{2} \pm \frac{1}{2\sqrt{3}}, \quad w_{\pm} = \frac{1}{2}.$$

At each path point the gas-pressure EOS partial is converted to the complete total-pressure composition coefficient with

$$\chi_{X_j}(\lambda) = \left. \frac{\partial \ln P}{\partial x_j} \right|_{\rho, T, \lambda} = \frac{P_{\text{gas}}(\lambda)}{P(\lambda)} \left. \frac{\partial \ln P_{\text{gas}}}{\partial x_j} \right|_{\rho, T, \lambda}.$$

The stored `chiX_face` coefficient is either the default face value or, in the optional Gauss mode, the path average

$$\bar{\chi}_{X_j} = \sum_{\pm} w_{\pm} \chi_{X_j}(\lambda_{\pm}).$$

The implicit Brunt composition term currently used by star is

$$B_f^{\text{implicit}} = \frac{\sum_j \chi_{X_j,f} (x_{j,k} - x_{j,k-1})}{\chi_{T,f} (\ln P_{k-1} - \ln P_k)} - \frac{\chi_{\rho,f} (\ln M_{\text{corr},k-1} - \ln M_{\text{corr},k})}{\chi_{T,f} (\ln P_{k-1} - \ln P_k)}.$$

This replaces the old MHM value difference

$$B_f^{\text{old}} = \frac{\ln P(\rho_f, T_f, x_k) - \ln P(\rho_f, T_f, x_{k-1})}{\chi_{T,f} (\ln P_{k-1} - \ln P_k)} + \text{mass-correction term}.$$

The key change is that the composition sensitivity comes from analytic EOS partials at the face, not from subtracting two EOS calls at the cell compositions. In `star/private/implicit_brunt.f90`, this coefficient is named `chiX_face`; it is kept fixed in the current Jacobian so this slice does not require second EOS composition derivatives.

Scalar AD through MLT/TDC.

The implicit path keeps species-sized derivatives out of MLT/TDC. Instead it passes one scalar autodiff Brunt composition term,

$$\text{gradL_composition_term}^{\text{ad}} \equiv B_f^{\text{ad}}.$$

MLT/TDC computes

$$D_{\text{mix,implicit}}^{\text{ad}} = D_{\text{MLT/TDC}}^{\text{ad}} \left(\nabla_{\text{rad}}^{\text{ad}}, \nabla_{\text{ad}}^{\text{ad}}, B_f^{\text{ad}}, \dots \right).$$

The current component policy only uses this local AD coefficient when the mixing result is **convective_mixing** or **semiconvective_mixing**. Thus TDC convective regions enter through the same **mlt_d_ad** storage as ordinary MLT, semiconvection remains implicit through the semiconvective AD return path, and thermohaline, overshoot, rotation, minimum mixing, **other_D_mix**, and later post-processing stay in **Dmix_explicit**.

The final real coefficient remains

$$D_{\text{mix}} = \text{val} \left(D_{\text{mix,implicit}}^{\text{ad}} \right) + D_{\text{mix,explicit}}.$$

The scalar AD slot **i_xtra2_00** is presently used as the local seed for $\partial D_{\text{mix}} / \partial B_f$. The species chain rule that will be needed in the isotope residual is therefore

$$\frac{\partial D_{\text{mix}}}{\partial x_{\ell,k}} = \frac{\partial D_{\text{mix}}}{\partial B_f} \frac{\partial B_f}{\partial x_{\ell,k}}, \quad \frac{\partial D_{\text{mix}}}{\partial x_{\ell,k-1}} = \frac{\partial D_{\text{mix}}}{\partial B_f} \frac{\partial B_f}{\partial x_{\ell,k-1}}.$$

The storage and scalar AD path now exist. **brunt_B_ad** carries structure derivatives through the pressure denominator and the face χ -terms. The face EOS composition coefficient $\chi_{X_j,f}$ is intentionally treated as a current-iteration coefficient for the species-indexed chain rule. That is not a missing first EOS partial. Differentiating $\chi_{X_j,f}$ itself would require second composition derivatives of the EOS pressure partials, which are out of scope for this implementation slice.

The **i_xtra2_00** seed is only an internal way to read $\partial D_{\text{mix}} / \partial B_f$. It is not a real solver variable: **star_utils:unpack_residual_partials** does not map the **xtra2** slots into the matrix. The actual species Jacobian receives the Brunt-composition coupling explicitly through **d_sig_dxa_m1(:, :)** and **d_sig_dxa_00(:, :)**.

Define

$$A = \ln P_{k-1} - \ln P_k, \quad C = \chi_{T,f}, \quad S = \sum_i \chi_{X_i,f} (X_{i,k} - X_{i,k-1}),$$

$$R = \chi_{\rho,f} (\ln M_{\text{corr},k-1} - \ln M_{\text{corr},k}), \quad B_f = \frac{S - R}{AC}.$$

Here $\chi_{X_i,f}$ is the coefficient stored in **chiX_face**. By default it is evaluated at the face composition; in the optional Gauss mode it is the two-point path average. The Jacobian treats this coefficient as fixed for the current Newton iteration.

The implemented first pass stores

$$\frac{\partial B_f}{\partial X_{\ell,k-1}} = \frac{S_{\ell,k-1} - R_{\ell,k-1}}{AC} - B_f \left(\frac{A_{\ell,k-1}}{A} + \frac{C_{\ell,k-1}}{C} \right),$$

$$\frac{\partial B_f}{\partial X_{\ell,k}} = \frac{S_{\ell,k} - R_{\ell,k}}{AC} - B_f \left(\frac{A_{\ell,k}}{A} + \frac{C_{\ell,k}}{C} \right).$$

Here

$$S_{\ell,k-1} = -\chi_{X_{\ell},f}, \quad S_{\ell,k} = \chi_{X_{\ell},f},$$

$$C_{\ell,k-1} = (1 - \alpha) \left. \frac{\partial \chi_T}{\partial X_{\ell}} \right|_f, \quad C_{\ell,k} = \alpha \left. \frac{\partial \chi_T}{\partial X_{\ell}} \right|_f.$$

Unless the hydrostatic fallback is used for A ,

$$A_{\ell,k-1} = \left. \frac{\partial \ln P}{\partial X_{\ell}} \right|_{k-1}, \quad A_{\ell,k} = - \left. \frac{\partial \ln P}{\partial X_{\ell}} \right|_k.$$

For the mass-correction term the current branch includes both the face χ_{ρ} derivative and the direct $\Delta \ln M_{\text{corr}}$ derivative. With

$$M_{\ell,k} = \frac{1}{M_{\text{corr},k}} \left(\frac{W_{\ell}}{A_{\ell}} - M_{\text{corr},k} \right),$$

the implemented terms are

$$R_{\ell,k-1} = \Delta \ln M_{\text{corr}} (1 - \alpha) \left. \frac{\partial \chi_{\rho}}{\partial X_{\ell}} \right|_f + \chi_{\rho,f} M_{\ell,k-1}, \quad R_{\ell,k} = \Delta \ln M_{\text{corr}} \alpha \left. \frac{\partial \chi_{\rho}}{\partial X_{\ell}} \right|_f - \chi_{\rho,f} M_{\ell,k},$$

Current implicit-Dmix Jacobian contribution.

`mix_info:get_convection_sigmas` keeps the existing real face coefficient

$$\sigma_k = \frac{\text{mix_factor } D_{\text{mix},k} (4\pi r_k^2 \rho_{f,k})^2}{m_\star (dq_k + dq_{k-1})/2}.$$

When `implicit_diffusion_flag` is true it also stores

$$\frac{\partial \sigma_k}{\partial X_{\ell,k-1}} = \frac{\text{mix_factor } (4\pi r_k^2 \rho_{f,k})^2}{m_\star (dq_k + dq_{k-1})/2} \frac{\partial D_{\text{mix},k}^{\text{implicit}}}{\partial B_k} \frac{\partial B_k}{\partial X_{\ell,k-1}},$$

and the analogous derivative with respect to $X_{\ell,k}$. If a σ limiter replaces the value by a fixed cap, the derivative is zeroed; if the high- T_c limiter scales σ , the derivative is scaled by the same factor.

The same routine also stores $\sigma_k^{\text{implicit,ad}}$ in `sig_implicit_ad(k)`. Its value is mirrored to the final real σ_k , but its derivatives come only from the implicit coefficient:

$$\sigma_k^{\text{implicit,ad}} = \frac{\text{mix_factor } D_{\text{mix},k}^{\text{implicit,ad}} (4\pi r_k^2 \rho_{f,k}^{\text{ad}})^2}{m_\star (dq_k + dq_{k-1})/2}.$$

The existing mixing rate in `star/private/mix_info.f90:set_dxd_t_mix` remains

$$\text{dxd_t_mix}_{j,k} = \frac{-\sigma_{k+1}(X_{j,k} - X_{j,k+1}) + \sigma_k(X_{j,k-1} - X_{j,k})}{dm_k}.$$

`star/private/hydro_chem_eqns.f90:do1_chem_eqns` already had the constant- σ , same-species derivatives. The new dev path adds the cross-species terms

$$\begin{aligned} \frac{\partial \text{dxd_t_mix}_{j,k}}{\partial X_{\ell,k-1}} &\supset \frac{X_{j,k-1} - X_{j,k}}{dm_k} \frac{\partial \sigma_k}{\partial X_{\ell,k-1}}, \\ \frac{\partial \text{dxd_t_mix}_{j,k}}{\partial X_{\ell,k}} &\supset \frac{X_{j,k-1} - X_{j,k}}{dm_k} \frac{\partial \sigma_k}{\partial X_{\ell,k}} - \frac{X_{j,k} - X_{j,k+1}}{dm_k} \frac{\partial \sigma_{k+1}}{\partial X_{\ell,k}}, \\ \frac{\partial \text{dxd_t_mix}_{j,k}}{\partial X_{\ell,k+1}} &\supset -\frac{X_{j,k} - X_{j,k+1}}{dm_k} \frac{\partial \sigma_{k+1}}{\partial X_{\ell,k+1}}. \end{aligned}$$

These are added to the existing isotope equation

$$s\% \text{equ}(s\% \text{nvar_hydro} + j, k) = \frac{\text{dxd_t_mix}_{j,k} + \text{dxd_t_nuc}_{j,k} - \text{dxd_t_actual}_{j,k}}{\text{eqn_scale}_{j,k}},$$

so no second composition equation is introduced.

For the structure part of this Jacobian, the lower face is already in the local equation- k stencil. The upper face is computed at $k+1$, so it must be shifted before it is used in the equation at k :

$$\sigma_{k+1}^{\text{ad}}|_{\text{eqn } k} = \text{shift_p1}(\text{sig_implicit_ad}(k+1)).$$

The residual AD contribution is therefore stored from

$$\frac{\sigma_k^{\text{ad}}(X_{j,k-1} - X_{j,k}) - \text{shift_p1}(\sigma_{k+1}^{\text{ad}})(X_{j,k} - X_{j,k+1})}{dm_k \text{eqn_scale}_{j,k}}.$$

This is the same tridiagonal convention as other MESA AD residuals: m1, 00, and p1 are always relative to the equation row currently being assembled.

Legend

Current MESA behavior. Existing code or equations already present in the repository. These are source-level facts to preserve or use.

New EOS implementation. Analytic EOS-side derivatives to add, mostly in `eos/private`. This is the main work needed to remove finite-difference composition partials for Skye/eosDT-covered cells.

New or changed star-side use. Changes in `star/private` that consume the new EOS partials, especially Brunt and possible coupled implicit diffusion work.

Verification only. Finite differences are acceptable here as tests of analytic derivatives, but not as production replacement partials.

Design decision. A choice that should be made explicitly before implementation broadens.

Risk or nonlinear boundary. Active-set changes, blend kinks, phase branches, and API expansion points.

Effort Summary

Area	Color	Effort	Why
Composition helper and moment derivatives	New EOS	S	Mostly wraps formulas already in <code>chem_lib</code> .
Skye ideal-ion partials	New EOS	M	Direct analytic chain rule through active-set number fractions.
Skye electron partials	New EOS	M	HELM derivative tables already contain the needed density derivatives.
Skye Coulomb partials	New EOS	L/XL	Many scalar composition inputs and phase/branch logic.
eosDT blend $d\alpha/dX_i$	New EOS	M	Plumbing plus PC, OPAL/SCVH, and full-Skye polygon alpha derivatives are in.
Complete Brunt EOS-composition form	Star-side	M	Dev branch now has a face-EOS-partial path and first-pass dB/dx ; second EOS composition derivatives are intentionally out of scope.
Coupled implicit diffusion	Star-side	XL	First <code>Dmix_implicit</code> Jacobian slice is in; microscopic diffusion remains a separate solver-structure project.

Area	Color	Effort	Why
Rotation/other transport compatibility	Star-side	M	Default-off controls should preserve current operator-split workflows.
Finite-difference checks	Verification	M	Needed to validate; not production behavior.

Current Contract

EOS public export.

`num_eos_d_dxa_results = 2`

Only the first two composition derivative rows are reliably returned to `star`:

$$d_dxa(i_lnP_{gas}, j) = \left. \frac{\partial \ln P_{gas}}{\partial X_j} \right|_{\rho, T}, \quad d_dxa(i_lnE, j) = \left. \frac{\partial \ln E}{\partial X_j} \right|_{\rho, T}.$$

Internally `eos/public/eos_lib.f90` already allocates `num_eos_basic_results` rows and then copies only the first two rows back to `star`.

Composition normalization and gauge.

MESA composition variables satisfy

$$S_X \equiv \sum_i X_i = 1, \quad \sum_i \delta X_i = 0$$

for physical perturbations. A species-basis derivative vector has gauge freedom: adding a constant to all species partials does not change any constrained derivative or any contraction with a zero-sum gradient.

$$\sum_i \left(\frac{\partial Q}{\partial X_i} + C \right) \delta X_i = \sum_i \frac{\partial Q}{\partial X_i} \delta X_i.$$

`star/private/star_solver.f90` currently uses a sink projection:

$$D_j^{\text{sink}} Q = \frac{\partial Q}{\partial X_j} - \frac{\partial Q}{\partial X_s}.$$

Composition Moments

Existing equations in `chem/public/chem_lib.f90`.

Let

$$Y_i = \frac{X_i}{A_i}, \quad Y = \sum_i Y_i, \quad \bar{A} = \frac{S_X}{Y}.$$

For a charge moment g_i ,

$$M_g = \bar{A} \sum_i \frac{X_i}{A_i} g_i.$$

MESA uses $g_i = Z_i$, Z_i^2 , and `chem_isos%Z53` for $\bar{Z}$, $\bar{Z}^2$, and $\bar{Z}^{5/3}$. The implemented derivatives are

$$\begin{aligned} \frac{\partial \bar{A}}{\partial X_j} &= \frac{\bar{A}}{A_j S_X} (A_j - \bar{A}), \\ \frac{\partial M_g}{\partial X_j} &= \frac{\bar{A}}{A_j S_X} (g_j - M_g), \\ m_c &= \sum_i X_i \frac{W_i}{A_i}, \quad \frac{\partial m_c}{\partial X_j} = \frac{W_j}{A_j} - m_c. \end{aligned}$$

Add reusable EOS helper.

Add a private helper, probably `eos/private/eos_composition_partials.f90`, that returns

$$\bar{A}_j, \quad \bar{Z}_j, \quad Y_{e,j}, \quad \bar{Z}^2_j, \quad \bar{Z}^{5/3}_j, \quad m_{c,j},$$

plus active-set normalized number-fraction derivatives for Skye.

For a fixed active set,

$$y_i = \frac{X_i/A_i}{Y_{\text{active}}}, \quad Y_{\text{active}} = \sum_{m \in A} \frac{X_m}{A_m},$$

so the helper returns

$$y_{i,j} = \begin{cases} \frac{\delta_{ij}}{A_i Y_{\text{active}}} - \frac{y_i}{A_j Y_{\text{active}}}, & j \in A, \\ 0, & j \notin A. \end{cases}$$

For complete ionization,

$$Y_e = \frac{\bar{Z}}{\bar{A}} = \sum_i X_i \frac{Z_i}{A_i}, \quad Y_{e,j} = \frac{Z_j}{A_j} - Y_e.$$

Effort: **S**. These are direct formulas already consistent with `chem_lib`.

Skye Analytic Partial

Current Skye state.

`eos/private/skye.f90:Get_Skye_EOS_Results` calls `skye_eos(..., d_dxa, ierr)` and then sets

$$d_dxa = 0.$$

The value path already uses `auto_diff_real_2var_order3` for T and ρ , but not for arbitrary composition variables.

Core Skye strategy.

Do not introduce global arbitrary-size autodiff. For each species j , compute a companion object

$$F_j = \left. \frac{\partial F}{\partial X_j} \right|_{\rho, T},$$

where F_j itself is still an `auto_diff_real_2var_order3` object in T and ρ .

Skye's existing thermodynamics are

$$S = -\frac{\partial F}{\partial T}, \quad P_{\text{gas}} = \rho^2 \frac{\partial F}{\partial \rho}, \quad E = F + TS.$$

Therefore

$$S_j = -\frac{\partial F_j}{\partial T}, \quad P_{\text{gas},j} = \rho^2 \frac{\partial F_j}{\partial \rho}, \quad E_j = F_j + TS_j.$$

The packed outputs are

$$\frac{\partial \ln S}{\partial X_j} = \frac{S_j}{S}, \quad \frac{\partial \ln E}{\partial X_j} = \frac{E_j}{E}, \quad \frac{\partial \ln P_{\text{gas}}}{\partial X_j} = \frac{P_{\text{gas},j}}{P_{\text{gas}}}.$$

Effort: **M** for the framework, before Coulomb.

Ideal-ion term.

Skye's ideal-ion free energy is

$$F_{\text{ion}} = \frac{k_B T}{m_u \bar{A}} \Phi, \\ \Phi = \sum_i y_i \left[\ln \left(\frac{y_i n}{n_{Q,i}} \right) - 1 \right], \quad n = \frac{\rho}{m_u \bar{A}}, \quad n_{Q,i} = n_Q W_i^{3/2}.$$

With active-set normalized number fractions y_i ,

$$\Phi_j = \sum_i y_{i,j} \ln \left(\frac{y_i n}{n_{Q,i}} \right) - \frac{\bar{A}_j}{\bar{A}}, \\ F_{\text{ion},j} = \frac{k_B T}{m_u} \left(\frac{\Phi_j}{\bar{A}} - \frac{\Phi \bar{A}_j}{\bar{A}^2} \right).$$

Ion offsets use full-network number fractions:

$$F_{\text{off}} = \frac{\text{eV}}{m_u} \sum_i \tilde{y}_i I(Z_i), \quad F_{\text{off},j} = \frac{\text{eV}}{m_u} \sum_i \tilde{y}_{i,j} I(Z_i).$$

Effort: **M**. Main risk is keeping active-set and full-network normalizations distinct.

Electron term.

`compute_ideal_ele` evaluates the HELM free energy as

$$D = Y_e \rho, \quad F_e = Y_e f(T, D).$$

For

$$f_{a,b} = \frac{\partial^{a+b} f}{\partial T^a \partial D^b},$$

the direct electron composition companion is

$$F_{e,Y} = \frac{\partial F_e}{\partial Y_e} = f + \rho Y_e f_{0,1},$$

and its packed temperature-density derivatives are

$$\frac{\partial^{a+b} F_{e,Y}}{\partial T^a \partial \rho^b} = (b+1) Y_e^b f_{a,b} + \rho Y_e^{b+1} f_{a,b+1}.$$

the stored coefficient is

$$\frac{\partial^{a+b} F_e}{\partial T^a \partial \rho^b} = Y_e^{b+1} f_{a,b}.$$

The composition derivative is

$$\frac{\partial}{\partial X_j} \left[\frac{\partial^{a+b} F_e}{\partial T^a \partial \rho^b} \right] = Y_{e,j} [(b+1) Y_e^b f_{a,b} + \rho Y_e^{b+1} f_{a,b+1}].$$

The implemented `F_dye` object fills these derivatives through third order in (T, ρ) . The highest density-only slot requires $f_{0,4}$, so `helm_polynomials.f90` now includes fourth derivatives of the quintic Hermite basis functions. Current Skye overwrites HELM `xnefer` with complete-ionization `compute_xne`, so

$$\ln \text{free}_e = \ln Y_e, \quad \frac{\partial \ln \text{free}_e}{\partial X_j} = \frac{Y_{e,j}}{Y_e},$$

away from the existing $\max(10^{-99}, x_{\text{nefer}})$ clamp.

Effort: **M**. Most table derivatives already exist.

Coulomb term.

Treat Coulomb as a chain-rule problem over composition-dependent scalar inputs:

$$F_C = G(T, \rho, u_1, \dots, u_m), \quad F_{C,j} = \sum_a \frac{\partial G}{\partial u_a} \frac{\partial u_a}{\partial X_j}.$$

The implementation should use a Skye-local companion type carrying (u, u_j) for one composition perturbation at a time, where each entry is an `auto_diff_real_2var_order3` in T, ρ . Looping over species gives arbitrary network size without parameterized derived types. This avoids hand-expanding every Coulomb fit.

The first implementation slice is `eos/private/skye_composition_ad.f90`. For a unary operation $z = f(u)$, it stores

$$z = f(u), \quad z_j = f'(u)u_j,$$

where both z and z_j are still third-order autodiff objects in (T, ρ) . For multiplication,

$$(uv)_j = u_jv + uv_j,$$

and similarly for division and powers. This is enough to reuse the Coulomb fit algebra while carrying one species derivative at a time.

The active-set number-fraction derivative needed by the Coulomb sums is now shared with the ideal-ion term through `get_active_number_fractionpartials`:

$$\frac{\partial y_i}{\partial X_j} = -\frac{y_i}{A_j Y_{\text{active}}} + \delta_{ij} \frac{1}{A_i Y_{\text{active}}}, \quad Y_{\text{active}} = \sum_{i \in \text{active}} \frac{X_i}{A_i}.$$

The current implementation wires this companion path through the internal full-partials EOS route. Legacy `eosDT_get` calls `Get_eosDT_Results` with `include_compositionpartials=.false.`; the dev full-row call `eosDT_get_full_dxa` passes true. This keeps the species loop and companion Coulomb algebra out of ordinary Skye calls.

The analytic Coulomb derivative itself is implemented in `skye_coulomb:nonideal_corrections_dxa`. For each original isotope X_j , `skye_eos` constructs

$$x_{\text{ne},j} = \rho N_A Y_{e,j},$$

then propagates

$$r_s = \left(\frac{3}{4\pi x_{\text{ne}} a_0^3} \right)^{1/3}, \quad \Gamma_e = \frac{e^2}{a_0 k_B T r_s},$$

and the unit conversion

$$k_T = \frac{k_B T}{A m_u}, \quad (k_T)_j = -k_T \frac{\bar{A}_j}{A}.$$

The companion path currently covers:

$$F_C = k_T \text{softmin} [F_{\text{liq}}(y_i, r_s, \Gamma_e), F_{\text{sol}}(y_i, r_s, \Gamma_e)],$$

including `EXCOR7`, linear mixing entropy, OCP liquid/solid terms, screening corrections, liquid/solid mixing corrections, Skye's existing Gamma-limit extrapolation, and the latent-heat phase softmin. The returned $F_{C,j}$, phase_j , and latent derivatives are added in `skye.f90` before `pack_compositionpartials`.

Effort: **L/XL**. This is the largest EOS-side piece because of liquid/solid fits, mixing entropy, phase softmin, and branch behavior. The next work is compile-level review and finite-difference verification; branch switch derivatives are still the selected-branch derivatives, matching the existing Skye value path.

Skye branch risks.

Derivatives are only smooth while the active set is unchanged:

$$X_i > \text{mass_fraction_limit_for_Skye}.$$

At exact active-set boundaries, min / max boundaries, and phase branch boundaries, use the derivative of the selected value branch and avoid those points in finite-difference tests.

eosDT and Blends

Current table mapping.

`eos/private/eosdt_eval.f90:get1_for_eosdt` maps table derivatives by charge:

$$\frac{\partial R}{\partial X_j} = \frac{\partial R}{\partial X_H} H_j + \frac{\partial R}{\partial Z} M_j,$$

with

$$H_j = \begin{cases} 1, & Z_j = 1, \\ 0, & \text{otherwise,} \end{cases} \quad M_j = \begin{cases} 1, & Z_j \neq 1 \text{ and } Z_j \neq 2, \\ 0, & \text{otherwise.} \end{cases}$$

Helium gets zero table derivative because the tables use $Y = 1 - X_H - Z$.

Implemented blend derivative.

For

$$R = \alpha R_1 + (1 - \alpha) R_2,$$

the complete composition derivative is

$$R_j = \alpha R_{1,j} + (1 - \alpha) R_{2,j} + \alpha_j (R_1 - R_2).$$

Current `Do_Blend` has the first two terms only. Add α_j , including the existing quintic smoothing:

$$\alpha = h(a_0) = 10a_0^3 - 15a_0^4 + 6a_0^5,$$

$$\alpha_j = h'(a_0) a_{0,j}, \quad h'(a_0) = 30(a_0 - 1)^2 a_0^2.$$

`eos/private/eosdt_support.f90:Do_Blend` now applies this term. `eos/private/eosdt_eval.f90:combine_for_eosdt` accepts an optional α_j vector so component alphas can opt in without changing legacy call sites. For PC Gamma-limit blends,

$$\log \Gamma_{e,0} = \log_{10} \left[\frac{q_e^2}{k_B} \left(4\pi N_A \frac{\bar{Z}}{3\bar{A}} \right)^{1/3} \right],$$

so

$$\frac{\partial \log \Gamma_{e,0}}{\partial X_j} = \frac{1}{3 \ln 10} \left(\frac{\bar{Z}_j}{\bar{Z}} - \frac{\bar{A}_j}{\bar{A}} \right).$$

In the PC $\log\text{-}\Gamma_e$ transition,

$$\alpha = \frac{\log \Gamma_{e,\text{all PC}} - \log \Gamma_e}{\log \Gamma_{e,\text{all PC}} - \log \Gamma_{e,\text{all HELM}}},$$

therefore

$$\alpha_j = - \frac{\partial \log \Gamma_{e,0} / \partial X_j}{\log \Gamma_{e,\text{all PC}} - \log \Gamma_{e,\text{all HELM}}}.$$

For the OPAL/SCVH high- Z HELM-reduction factor,

$$\alpha = \alpha_0 \frac{Z - Z_{\text{all OPAL}}}{Z_{\text{all HELM}} - Z_{\text{all OPAL}}},$$

so

$$\alpha_j = \frac{\alpha_0}{Z_{\text{all HELM}} - Z_{\text{all OPAL}}} \frac{\partial Z}{\partial X_j}, \quad \frac{\partial Z}{\partial X_j} = \begin{cases} 1, & Z_j \neq 1 \text{ and } Z_j \neq 2, \\ 0, & Z_j = 1 \text{ or } Z_j = 2. \end{cases}$$

For the full Skye polygon blend, the point

$$p = (\log \rho, \log T)$$

is fixed for a composition derivative, but some polygon vertices move with $\bar{A}$ and $\bar{Z}$. The implemented derivative keeps the current nearest edge fixed, matching the value branch used by `eos_blend:min_distance_to_polygon`. If

$$d(p, \Omega)$$

is the signed distance to the Skye polygon and

$$\alpha = \text{clip} \left(\frac{d}{w}, 0, 1 \right),$$

then inside the active transition band

$$\alpha_j = \frac{1}{w} \left(\frac{\partial d}{\partial \bar{A}} \bar{A}_j + \frac{\partial d}{\partial \bar{Z}} \bar{Z}_j \right).$$

The derivative is zero outside the transition band after the same clipping used by the value path.

The simple Skye blend remains composition independent because it is a product of one-dimensional $\log T$ and $\log \rho$ ramps.

Star-Side Consumers

Current MHM pressure-difference approximation.

Current MESA computes the Brunt composition term in `star/private/brunt.f90:do_brunt_B_MHM_form` by calling the EOS twice at the same face (ρ_f, T_f) , but with neighboring cell compositions:

$$\ln P_1 = \ln P_{\text{eos}}(\rho_f, T_f, X_k), \quad \ln P_2 = \ln P_{\text{eos}}(\rho_f, T_f, X_{k-1}).$$

Then

$$B_{\text{MHM}} = \frac{\ln P_1 - \ln P_2}{(\ln P_{\text{eos},k-1} - \ln P_{\text{eos},k})\chi_{T,f}}.$$

Linearizing the numerator gives

$$\ln P_1 - \ln P_2 \approx \sum_j \chi_{X_j,f} (X_{j,k} - X_{j,k-1}),$$

so

$$B_{\text{MHM}} \approx -\frac{1}{\chi_{T,f}} \sum_j \chi_{X_j,f} \frac{X_{j,k-1} - X_{j,k}}{\ln P_{\text{eos},k-1} - \ln P_{\text{eos},k}}.$$

This is the first-order linearization of the complete EOS-composition form, but the current value calculation gets the EOS composition contraction from a finite pressure difference. After `brunt_B` is computed, `hydro_vars.f90` stores it in `gradL_composition_term`, and `turb_support.f90` uses

$$\nabla_L = \nabla_{\text{ad}} + \text{gradL_composition_term}.$$

Complete EOS-composition Brunt form.

The branch `origin/Brunt_B_from_eos_partials` wants the complete EOS-composition form

$$B_{\text{comp}} = -\frac{1}{\chi_T} \sum_j \chi_{X_j} \frac{dX_j}{d \ln P}.$$

The EOS partial itself must be returned by the EOS at the face state:

$$g_{j,f} = \left. \frac{\partial \ln P_{\text{gas}}}{\partial X_j} \right|_{\rho_f, T_f, X_f}.$$

This is the default implementation. It should not be approximated by finite-differencing EOS calls with neighboring cell compositions. The optional Gauss mode instead averages the same analytic EOS partial along

$$X_j(\lambda) = (1 - \lambda)X_{j,k-1} + \lambda X_{j,k}$$

and stores

$$\bar{g}_j = \sum_{\pm} w_{\pm} \left. \frac{\partial \ln P_{\text{gas}}}{\partial X_j} \right|_{\rho_f, T_f, X(\lambda_{\pm})}.$$

Since MESA currently returns gas-pressure partials, the total-pressure conversion must be applied at the same state. In the default path,

$$\chi_{X_j,f} = \frac{P_{\text{gas},f}}{P_{\text{eos},f}} g_{j,f}.$$

In the optional Gauss path,

$$\bar{\chi}_{X_j} = \sum_{\pm} w_{\pm} \frac{P_{\text{gas}}(\lambda_{\pm})}{P_{\text{eos}}(\lambda_{\pm})} \left. \frac{\partial \ln P_{\text{gas}}}{\partial X_j} \right|_{\rho_f, T_f, X(\lambda_{\pm})},$$
$$\chi_T = \frac{P_{\text{gas}}}{P_{\text{eos}}} \left. \frac{\partial \ln P_{\text{gas}}}{\partial \ln T} \right|_{\rho, X} + \frac{4P_{\text{rad}}}{P_{\text{eos}}}.$$

The remaining finite difference is the spatial composition gradient, not an EOS composition partial:

$$\frac{dX_j}{d \ln P} \approx \frac{X_{j,k-1} - X_{j,k}}{\ln P_{\text{eos},k-1} - \ln P_{\text{eos},k}}.$$

Effort: **M**, after analytic EOS partials exist.

Implicit diffusion coupling.

Current element diffusion is implicit inside `star/private/diffusion.f90:do_solve_diffusion`, but it is operator-split from the hydro Newton solve. MESA already has one hydro composition equation per isotope in `star/private/hydro_chem_eqns.f90:do1_chem_eqns`; coupled microscopic diffusion should add a rate to that equation, not create a second species equation.

In current term names,

$$\begin{aligned} \text{dxdt_actual}_{j,k} &= \frac{X_{j,k}^{n+1} - X_{j,k}^n}{\Delta t}, & \text{dxdt_expected}_{j,k} &= \text{dxdt_mix}_{j,k} + \text{dxdt_nuc}_{j,k}, \\ s\%equ(i,k) = R_{j,k}^X &= \frac{\text{dxdt_expected}_{j,k} - \text{dxdt_actual}_{j,k}}{\text{eqn_scale}_{j,k}} = 0, & i &= s\%nvar_hydro + j. \end{aligned}$$

For the current `Dmix_implicit` slice, no second composition equation and no separate rate array are introduced. The rate is still `dxdt_mix`; the new work is the additional Jacobian from $\partial\sigma/\partial X$, described above in the current implementation section.

A future coupled microscopic diffusion term would instead add another rate to the same isotope equation:

$$\begin{aligned} \text{dxdt_expected}_{j,k} &= \text{dxdt_mix}_{j,k} + \text{dxdt_nuc}_{j,k} + \text{dxdt_diff}_{j,k}, \\ \text{dxdt_diff}_{j,k} &= -\frac{\Phi_{j,k+1/2} - \Phi_{j,k-1/2}}{\Delta m_k}. \end{aligned}$$

To allow some transport to remain split while another part is coupled, accumulate the coefficient in two pieces:

$$D_{\text{mix}} = D_{\text{mix}}^{\text{explicit}} + D_{\text{mix}}^{\text{implicit}}.$$

Here explicit means outside the main hydro Newton block, or included as a frozen source with no new Jacobian. The implicit part supplies the new rate and Jacobian:

$$\text{dxdt_diff, implicit}_{j,k} = -\frac{\Phi_{j,k+1/2}^{\text{implicit}} - \Phi_{j,k-1/2}^{\text{implicit}}}{\Delta m_k}, \quad \frac{\partial R_{j,k}^X}{\partial X_{\ell,m}} \supset \frac{1}{\text{eqn_scale}_{j,k}} \frac{\partial \text{dxdt_diff, implicit}_{j,k}}{\partial X_{\ell,m}}.$$

The mass-fraction constraint can be handled by a sink species or by

$$R_k^\Sigma = \sum_j X_{j,k} - 1 = 0.$$

Effort: **XL**. EOS composition partials are prerequisite, not the whole implicit-diffusion implementation.

Transport component policy.

Default behavior should not change when the dev implicit-diffusion flag is off. When the flag is on, the intended component split is:

MLT	implicit
TDC	implicit
semiconvection	implicit
rotation	semi-implicit / explicit component
other non-semiconvective transport	semi-implicit / explicit component

turb_support quantities are updated per Newton iteration, so they can feed the current implicit coefficient without being a stale outer-loop source. Rotation modifies `s%D_mix` through `star/private/mix_info.f90:update_rotation_mixing_info`, but that contribution should remain outside `Dmix_implicit` unless a separate control explicitly promotes it.

Internally:

$$\begin{aligned} D_{\text{mix,implicit}} &= D_{\text{MLT}} + D_{\text{TDC}} + D_{\text{semiconv}}, \\ D_{\text{mix,explicit}} &= D_{\text{rot}} + D_{\text{other}} + \dots, \\ D_{\text{mix}} &= D_{\text{mix,implicit}} + D_{\text{mix,explicit}}. \end{aligned}$$

Here “explicit” includes the existing semi-implicit or split workflows: the coefficient may be refreshed each iteration, but it does not contribute the new coupled Brunt/species Jacobian terms unless it has been promoted to an implicit component. The exact implementation may need more than scalar D_{mix} arrays for multicomponent diffusion, but the split keeps the interface clear.

Any additional coupled transport source should return both its rate and Jacobian contribution. It should also conserve mass fraction in the solver basis:

$$\sum_j \text{dxdt_diff}_{j,k} = 0 \quad \text{or} \quad \sum_j \Phi_{j,k+1/2} = 0,$$

up to the chosen sink/projection convention.

Effort: **M** for controls and plumbing, larger if rotation’s angular-momentum coupling is pulled into the same Newton block.

Current EOS Numeric Validation

Implemented plotter check.

The local validation driver is

`eos/plotter/composition_partials/src/eos_composition_partials_check.f90.`

It evaluates the full analytic path

`eosDT_get_full_dxa`

and compares each row against a constrained finite difference that preserves $\sum_i X_i = 1$. With sink species $s = \text{he4}$, the checked analytic quantity is

$$A_{q,i} = \left. \frac{\partial q}{\partial X_i} \right|_{\rho,T} - \left. \frac{\partial q}{\partial X_s} \right|_{\rho,T}.$$

The finite-difference value is

$$F_{q,i}(h) = \frac{q(X_i + h, X_s - h) - q(X_i - h, X_s + h)}{2h}.$$

The driver uses the same Ridders/Neville extrapolation idea as `num/public/num_dfridr.dek`, but vectorized across all EOS result rows so each trial h only needs one plus and one minus EOS call:

$$F_{q,i}^{(m)}(h) = \frac{2^{m-1} F_{q,i}^{(m-1)}(h/\sqrt{2}) - F_{q,i}^{(m-1)}(h)}{2^{m-1} - 1}.$$

The CSV records the selected extrapolated derivative and its internal finite-difference error estimate.

The contour plotter also writes a focused DFRIDR grid for the rows most useful for this work:

$$q \in \{P_{\text{gas}}, \mu, \ln E\}.$$

Those pages intentionally keep the sink species distinct from the plotted direction. For the current diagnostic,

$$s = \text{h1}, \quad j \in \{\text{he4}, \text{c12}, \text{o16}\},$$

and the compared quantity is

$$D_j^{\text{h1}} q = \left. \frac{\partial q}{\partial X_j} \right|_{\rho,T} - \left. \frac{\partial q}{\partial X_{\text{h1}}} \right|_{\rho,T}.$$

For the gas pressure page the analytic value is formed as

$$D_j^{\text{h1}} P_{\text{gas}} = P_{\text{gas}} \left(\left. \frac{\partial \ln P_{\text{gas}}}{\partial X_j} \right|_{\rho,T} - \left. \frac{\partial \ln P_{\text{gas}}}{\partial X_{\text{h1}}} \right|_{\rho,T} \right).$$

The plotted DFRIDR error is

$$\log_{10} \frac{|A - F|}{\max(|A|, |F|, 10^{-14})},$$

where A is the analytic constrained partial and F is the Ridders finite-difference estimate. Failed or out-of-coverage EOS cells are masked before plotting, so they appear white and do not set the color scale.

For the default validation composition, $X = 0.70$ and $Z = 0.02$, the table EOS DFRIDR panels need a careful interpretation. OPAL/SCVH and FreeEOS are native $q(X, Z)$ tables in this path. With the fixed h1 sink,

$$D_{\text{he4}}^{\text{h1}} q = -q_X, \quad D_{\text{c12}}^{\text{h1}} q = D_{\text{o16}}^{\text{h1}} q = q_Z - q_X.$$

The he4 direction keeps Z fixed, and in pure OPAL/SCVH and FreeEOS cells it validates to near roundoff for the rows checked so far. The c12 and o16 directions move the finite-difference state through the $Z = 0.02$ table slice. At that slice, `eos/private/eosdt_eval.f90:Get1_eosdt_Results` returns the derivative of MESA's selected interpolation bracket, while a centered DFRIDR sample straddles the slice. The visible table-region metal-direction error is therefore partly a validation-stencil artifact, not by itself evidence that a free-energy reconstruction is required for this branch.

The useful next diagnostic is to add an off-table-slice composition case, e.g. $Z \neq 0.02$, and/or a one-sided table-stencil finite difference at exact table slices. A full free-energy reconstruction or free-energy-level table blend would be the larger project needed only if we require all OPAL/SCVH, FreeEOS, and blended derived rows to be mutually thermodynamically consistent as derivatives of one potential.

Sink species versus raw EOS coefficients.

The EOS path returns a coefficient vector $q_j \equiv (\partial q / \partial X_j)_{\rho, T}$. For any constrained composition perturbation,

$$\sum_j \delta X_j = 0,$$

the physically used differential is

$$\delta q = \sum_{j=1}^{N_{\text{spec}}} q_j \delta X_j.$$

This expression is unchanged if a row-wise constant C_q is added to every coefficient:

$$\sum_j (q_j + C_q) \delta X_j = \sum_j q_j \delta X_j + C_q \sum_j \delta X_j = \sum_j q_j \delta X_j.$$

That is why the finite-difference checker chooses a sink species only for the comparison basis,

$$D_j^s q = q_j - q_s,$$

while the EOS itself should not hard-code a global sink.

The implicit Brunt path follows the same rule. It uses all species in the face contraction

$$\Delta \ln P_X = \sum_{j=1}^{N_{\text{spec}}} \chi_{X,j}^{\text{face}} (X_{j,k} - X_{j,k-1}),$$

where

$$\chi_{X,j}^{\text{face}} = \frac{P_{\text{gas}}}{P_{\text{gas}} + P_{\text{rad}}} \left. \frac{\partial \ln P_{\text{gas}}}{\partial X_j} \right|_{\rho, T}.$$

Since both adjacent cells are normalized,

$$\sum_j (X_{j,k} - X_{j,k-1}) = 0,$$

this all-species sum is algebraically equivalent to any $N_{\text{spec}} - 1$ sink-species basis. No special sink is needed inside `star/private/implicit_brunt.f90`; the sink only appears in finite-difference tests and in star's solver-partial diagnostic projection.

X/Z table EOS mapping.

OPAL/SCVH and FreeEOS are native table functions of hydrogen and metal coordinates,

$$q = q(X, Z),$$

with helium represented by the remaining composition coordinate. The full per-isotope coefficient vector now maps the native table derivatives through the same raw table coordinates used by `chem_lib:basic_composition_info`:

$$\frac{\partial X}{\partial X_j} = I_H(j), \quad \frac{\partial Z}{\partial X_j} = -I_H(j) - I_{\text{He}}(j).$$

Therefore

$$\left. \frac{\partial q}{\partial X_j} \right|_{\rho, T} = \frac{\partial q}{\partial X} \frac{\partial X}{\partial X_j} + \frac{\partial q}{\partial Z} \frac{\partial Z}{\partial X_j}.$$

For hydrogen, helium, and explicit metals this gives

$$q_H = q_X - q_Z, \quad q_{\text{He}} = -q_Z, \quad q_{\text{metal}} = 0.$$

This is the raw gauge seen by the star equations. The metal coefficient is zero because the table Z coordinate is the residual $1 - X_H - X_{\text{He}}$, not the explicit sum of metal species. A constrained perturbation with an H sink still sees

$$D_{\text{He}}^H q = -q_X, \quad D_{\text{metal}}^H q = q_Z - q_X,$$

and the Brunt pressure contraction is unchanged because the raw-coordinate mapping differs from the normalized-coordinate mapping only by a constant added to all species in a row.

This is still an analytic derivative in MESA’s table sense: the table wrapper returns q_X and q_Z from the selected interpolation stencil, and the isotope coefficients above are exact chain-rule projections of those derivatives. It is not a finite-difference reconstruction of FreeEOS.

Generated artifacts:

```
eos/plotter/composition_partials/data/eos_composition_partials.csv
eos/plotter/composition_partials/data/eos_composition_partials_contours.csv
eos/plotter/composition_partials/data/eos_composition_partials_summary.txt
eos/plotter/composition_partials/figures/*.pdf
eos/plotter/composition_partials/figures/*.png
eos/plotter/composition_partials/figures/all_composition_partial_contours.pdf
```

Default EOS controls.

The first plotter pass intentionally uses a normal default EOS handle. This answers the immediate “is the default path okay?” question by testing the same eosDT selection a MESA run would get without a special EOS inlist. The fraction columns show that this first grid covers HELM/FreeEOS and Skye regions for the chosen composition. It does not activate OPAL/SCVH, PC, CMS, or ideal; those need forced-component or additional state/composition grids.

Summary of selected max relative errors from the first default-control run:

Row	cool T	dense T	hot T	rho sweep
lnPgas	4.0e-7	6.6e-7	4.9e-7	5.1e-7
lnE	2.7e-7	1.7e-8	1.1e-6	2.4e-5
mu	5.8e-9	5.6e-11	7.6e-11	8.4e-11
chiT	4.6e-6	2.2e-5	1.8e-6	1.2e-6
chiRho	1.9	5.2e-3	2.9e-4	5.9e-4
grad_ad	1.7	3.8e-3	4.8e-3	1.1e-2

The HELM μ failure found by the first run was fixed in `eos/private/eos_helm_eval.f90`. The remaining large cool-sweep failures are in HELM/FreeEOS derived rows, especially χ_ρ , ∇_{ad} , Γ_1 , and C_P . Their finite-difference error estimates are small in the worst cases, so this is not just Ridders noise. Those rows need a separate HELM/FreeEOS derivative audit before calling the full default-control EOS partial suite complete.

Immediate validation follow-up:

1. Add component-forced or state-targeted grids for OPAL/SCVH, PC, CMS, and ideal so every component path is checked explicitly.
2. Audit HELM composition derivatives for derived thermodynamic rows against direct *abar*, *zbar* finite differences.
3. Add local Brunt and implicit-Dmix Jacobian numeric checks after the EOS row-level issues are bounded.

Verification Plan

Finite differences are tests, not production partials.

Raw EOS-partial check:

$$\left. \frac{\partial f}{\partial X_j} \right|_{\rho, T} \approx \frac{f(X_j + \epsilon) - f(X_j - \epsilon)}{2\epsilon}.$$

Constrained star-side check with sink species *s*:

$$D_j^{\text{sink}} f \approx \frac{f(X_j + \epsilon, X_s - \epsilon) - f(X_j - \epsilon, X_s + \epsilon)}{2\epsilon}.$$

Brunt contraction check:

$$B_{\text{comp}} = -\frac{1}{\chi_T} \sum_j \chi_{X_j} \frac{dX_j}{d \ln P}, \quad \sum_j \frac{dX_j}{d \ln P} \approx 0.$$

Run these only after explicit permission to compile/run MESA or targeted EOS tests.

Implementation Order

1. Add the composition helper and source-level tests of its formulas.
2. Add Skye ideal-ion and electron `d_dxa` for `i_lnPgas` and `i_lnE`.
3. Compile-review and derivative-test the Skye-local Coulomb/phase companion path now wired through `nonideal_corrections_`.
4. Continue filling/auditing all internal Skye `d_dxa(1:nv,:)` rows, but keep the public `num_eos_d_dxa_results = 2` until the star-side API audit is explicit.
5. Continue eosDT blend `dalp`/`dX_i` audit beyond the PC Gamma-limit, OPAL/SCVH high-*Z*, and full-Skye polygon cases now wired.
6. Verify the complete Brunt EOS-composition contraction and the current-iterate `chiX_face` Jacobian treatment. Second EOS composition derivatives are out of scope for this slice.
7. Current policy is wired: MLT/TDC convective regions and semiconvection enter `Dmix_implicit`; rotation and other non-semiconvective transport remain in `Dmix_explicit` unless separately promoted.